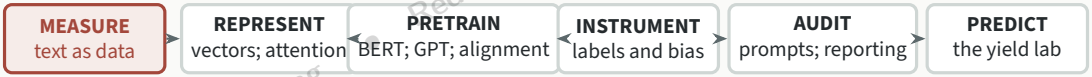
Session 5.1

Text as Economic Data: From Documents to Counts

Clean, tokenize, count: how a document becomes a vector a regression can use

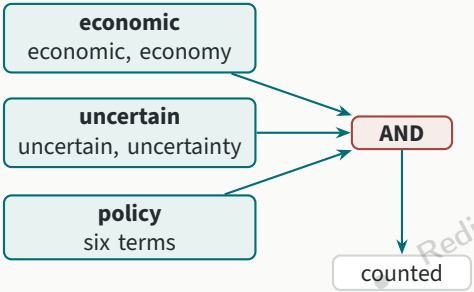
Based on Chapter 13 of Miao & Feng, *AI for Economists*



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

Reading Policy Uncertainty Out of Newspapers

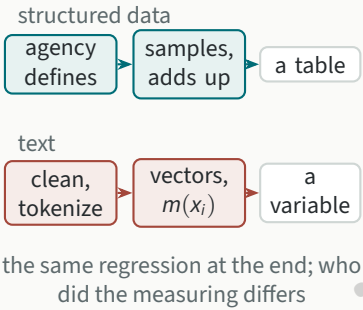


an article counts only if it hits all three groups; the same recipe gives China's index from 1949 (Davis, Liu & Sheng 2019)

- Baker, Bloom & Davis (2016): ten US newspapers, monthly since 1985; an article counts if it hits all three groups.
- $U_t = \frac{1}{10} \sum_p \frac{c_{pt}/N_{pt}}{\sigma_p}$, with c_{pt} hits among N_{pt} articles of paper p; the index is $100\, U_t / \bar{U}$.
- Checked against 12,009 hand-coded articles: machine and human indices correlate 0.86 by quarter (1985–2012), 0.93 by year (1900–2010).
- **No agency publishes this number: it was built from words and checked against readers.**

© Zhengyang Feng · Redistribution prohibited without permission

With Text, the Measurement Step Is Yours

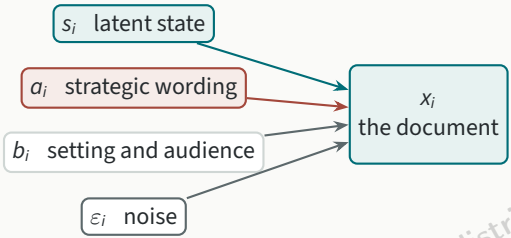


text is the	example	study
measure	policy uncertainty from newspapers	Baker, Bloom & Davis (2016)
outcome	FOMC speech after transcripts went public	Hansen, McMahon & Prat (2018)
treatment	central-bank talk and inflation expectations	Blinder et al. (2024)

Redistribution not allowed without permission
© Zheng Feng

- Prices arrive measured; statements, filings and job ads arrive as language. For a price index an agency defines, samples and adds up; for text nobody has.
- Text can be the regressor, the outcome or the treatment; this session builds the first.
- **With text, the researcher does the measuring.**

What a Text Measure Measures



two firms, the same risk s_i: one discloses, one deflects, so x_i differs through a_i

- The document is not the state: $x_i = g(s_i, a_i, b_i) + \varepsilon_i$. A measure $m(x_i)$ then enters $y_i = \beta\, m(x_i) + \boldsymbol{\gamma}^\top \mathbf{c}_i + u_i$.
- If the wording a_i shares a driver with u_i, $\hat{\beta}$ is biased, and a bigger corpus does not repair it.
- The measurement error $m(x_i) - s_i$ moves with s_i itself (a firm hiding a risk writes less about it): not classical, so no attenuation rule, and the sign of the bias is unknown.
- **Ask why $m(x_i)$ is exogenous before asking how well it is measured.**

© Zhigang Feng — Redistribution prohibited without permission

The Pipeline: Four Steps From a String to a Vector



	after step 1: clean	after step 2: tokenize	after step 3: count
EN	The central bank raised interest rates amid rising inflation.	The · central · bank · raised · interest · rates · amid · rising · inflation · .	one row of V counts, mostly 0
ZH	央行上调利率以遏制通胀。	央行 · 上调 · 利率 · 以 · 遏制 · 通胀 · 。	one row of V counts, mostly 0

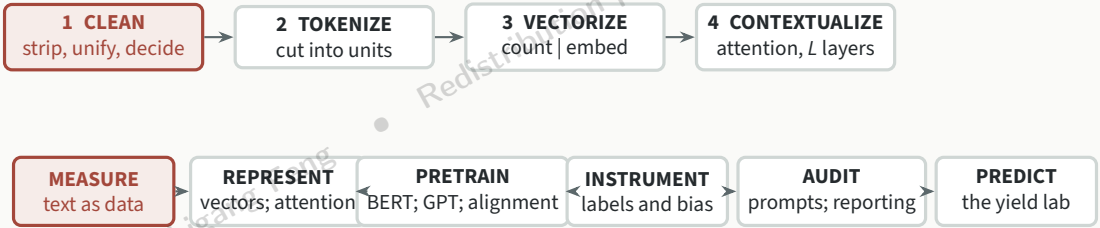
- Cleaning and tokenizing decide which strings count as one feature; vectorizing turns tokens into numbers. This deck counts; Deck 5.2 embeds, then adds context.
- **Different choices at any step give different regressors from the same documents.**

© Zhirang Feng Redistribution prohibited without permission

Session 5.1

Step 1 · Clean

What to remove, what to unify, and what must stay



Redistribution prohibited without permission
© Zhigang Feng

Cleaning, Worked on One Sentence

English, scraped from a web page

`<p>The central bank raised …</p>`

↓ strip mark-up and entities

The central bank raised …

Chinese, traditional characters

`<p>`央行上調利率以遏制通脹。`</p>`

↓ 繁 → 简: 調 → 调, 脹 → 胀

央行上调利率以遏制通胀。

ＣＰＩ同比上漲２．３％

↓ 全角 → 半角, 漲 → 涨

CPI 同比上涨 2.3%

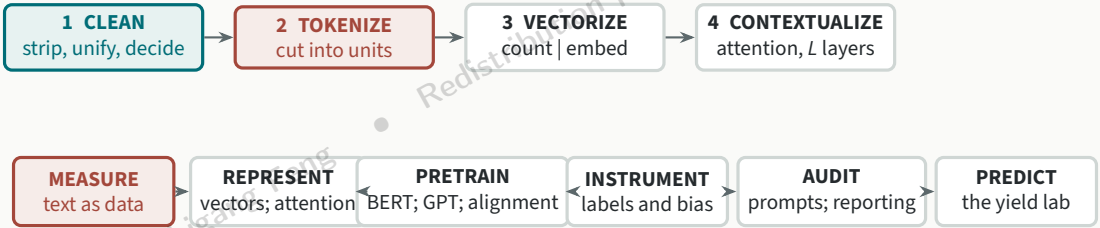
- Strip what is not text: mark-up, headers and footers, table debris, broken encodings. Unify what should be one string, or 通脹 and 通胀 are two words and ＣＰＩ never matches CPI.
- Decide by task: lower-casing merges the Fed with the verb *fed*; in finance %, ? and quotation marks carry information.
- **Write the rules as a script, and archive it with the data.**

© Yingxue Feng · Redistribution prohibited without permission

Session 5.1

Step 2 · Tokenize

Cut the string into the units you will count



Redistribution prohibited without permission
© Zhigang Feng

Tokens and the Vocabulary

English: spaces do most of the cutting

GDP	rose	by	2	%

Chinese: the boundaries must be inferred

央行	决定	下调	存款准备金率

央行决定下调存款准备金率: the central bank decides to cut the reserve requirement ratio

- A **corpus** is a collection of texts: a year of FOMC minutes, every annual report in an industry.
- A **token** is the smallest unit you count; **tokenization** cuts a string into tokens; the **vocabulary** is the set of distinct tokens.
- **Change the cut and you change every count that follows.**

EN	The	central	bank	raised	interest	rates	amid	rising	inflation	.
ZH	央行	上调	利率	以	遏制	通胀	。			

the running pair, cut into words

© Lijiang Feng Redistribution Prohibited without permission

Chinese Segmentation Is a Labelling Problem

研究生命科学

研究 | 生命科学 — study | life science

研究生 | 命科学 — graduate student | ?

行业政策支持

行业政策 | 支持 — head word: support

行业 | 政策支持 — head word: policy support

one label per character: B begins a word, E ends it

央	行	上	调	利	率
B	E	B	E	B	E

- Chinese characters run together, so segmentation is **sequence labelling**: for each character, decide whether a word ends after it.
- 研究生命科学 reads as *study life science*, or as nonsense if the cut falls after 生.
- 行业政策支持 splits two sensible ways with different head words: support for an industry policy, or policy support for an industry.
- **A wrong boundary changes what the sentence is about.**

© Zhigeng Geng
Redistribution prohibited without permission

Domain Compounds Need a User Dictionary

货币政策委员会, the monetary policy committee

general dictionary

货币 | 政策委员会 head word lost

with a user dictionary

货币政策委员会 kept whole

entries to add first:

社会融资规模 | 宏观审慎监管框架

- Segmenters such as Jieba, THULAC and LTP combine a dictionary with a statistical or neural labeller.
- Domain compounds are missing from general dictionaries, and a wrong cut changes the head word.
- The fix is a **user dictionary** listing terms such as 社会融资规模 (aggregate financing), so they stay whole.
- **Add the domain terms before you count, and publish the list.**

© Zhigang Feng · Redistribution prohibited without permission

Stop Words: Drop the Glue, Keep the Signal

usually dropped

the is of 的 了 是

keep: they carry the signal

not up down more less

因此 therefore 然而 however

- **Stop words**: frequent function words with little topical content (the, is, of; 的, 了, 是).
- General lists also delete up, down, more and less, the direction of a macro signal, and *not*, which flips a sentence; 因此 and 然而 carry the logic.
- Domain lists differ too: *liability* is negative in a general sentiment list and a neutral line item in accounts (Loughran & McDonald 2011).
- **A general stop list can delete the signal itself.**

© Zhigang Feng
Redistribution prohibited without permission

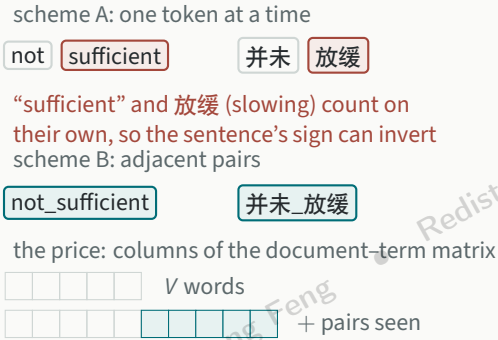
Stems, Lemmas, and a Language Without Inflection



- **Stemming** chops suffixes by rule, so investing, invests and invested all become *invest*; it also turns studies into *studi*, a non-word.
- **Lemmatization** returns the dictionary form using the part of speech: better → good, were → be.
- 上调 (raise) never changes form: tense sits in separate words, 已上调 (has raised) and 将上调 (will raise).
- **English must choose stem or lemma; Chinese has no need for either.**

© Zhigang Feng Redistribution prohibited without permission

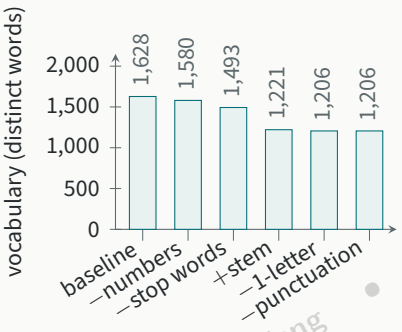
n-Grams: One Boundary Can Flip the Sign



- An **n-gram** treats n adjacent tokens as one feature: “not sufficient” stays hawkish; split, it reads as reassuring.
- Which pairs count is a rule we set: a minimum count, a collocation score, or a phrase list (Hassan et al. 2019 build one from political texts).
- The price: V words allow V^2 pairs, and every pair seen adds a column.
- **A pair is a feature when a rule says so; report the rule.**

© Zhigang Feng
Redistribution prohibited without permission

Six Pipelines, Five Vocabularies



step added	what it does	example
baseline	lower-case, split	Fed → fed
−numbers	drop numerals	“5-1/4” gone
−stop words	drop stop words	116: the, of, *but*
+stem	strip suffixes	inflation → infl
−1-letter	drop 1-letter tokens	stray letters
−punctuation	nothing left to drop	none

© Zhaonan Feng · Redistribution permitted without permission

- Each bar: the distinct words left in the 149 statements after every step up to it, 1,628 → 1,206 (−26%); the last two coincide, the tokenizer never emits punctuation.
- Each step loses something: “moderated *but* remains elevated” loses its turn with *but*.
- **Report your pipeline; show the result survives a change** (Denny & Spirling 2018).

Why Subwords

a word split into reusable pieces

macroeconomics → macro | economics

not among the lab's 1,628 words, yet spelled by its byte-pair pieces:

di·s·infl·a·tion·ar·y disinflationary

a policy term coined after the dictionary:

专精特新 → 专 | 精 | 特 | 新

- Word-level tokenizers fail twice: an unseen word gets no representation, and the vocabulary grows with every new form.
- **Subword tokenization** keeps reusable fragments, so firm names, tickers and new policy terms such as 专精特新 are spelled from known pieces.
- **With byte-level subwords, every string has a representation.**

© Zhigang Feng Redistribution prohibited without permission

Why Learn the Pieces From Counts

one word, four vocabulary sizes V. Top: every string spelled, but long, and no unit means anything alone

V

$V = 256$ bytes
d·i·s·i·n·f·l·a·t·i·o·n·a·r·y 15 tokens

letters + 200 merges (the lab's BPE)
di·s·infl·a·tion·ar·y 7

cl100k, $V \approx 100{,}000$
disin·flation·ary 3

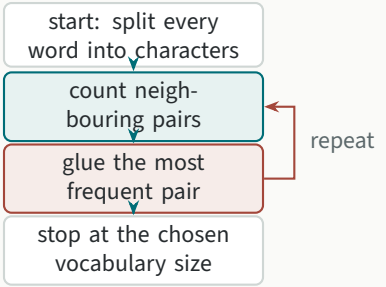
whole words, the statements' 1,628
disinflationary no slot

bottom: short, one unit one meaning, but the unseen has no slot

- Letters alone spell anything, but a statement becomes thousands of units, none meaning anything by itself.
- Whole words mean something, but an unseen word has no slot and the list never ends.
- In between, V is a choice: which strings get a slot? A language model learns one vector per slot, from that token's occurrences, so a slot pays only for a string that recurs. Counts know which.
- **Let the corpus's own counts pick the strings worth a slot.**

© Zhebin Feng Redistribution prohibited without permission

Byte-Pair Encoding: Learn the Pieces From Counts



通胀 $\times 5$, 通胀率 $\times 3$, 利率 $\times 4$ (inflation, inflation rate, interest rate)

start	通	胀	通	胀	率	利	率	
merge 1	通胀		通胀	率		利	率	通+胀: $5+3=8$
merge 2	通胀		通胀	率		利率		利+率: 4; 通胀+率: 3

a new word replays the merges: 通胀目标 (inflation target) → 通胀 · 目 · 标

© Zhiyang Feng · Redistribution prohibited without permission

- Goal: the recurring strings get a slot; any string stays spellable (Sennrich, Haddow & Birch 2016).
- What we do: start from characters; count neighbouring pairs; glue the most frequent pair (the merge that saves the most tokens); count again; repeat to the chosen size.
- **Frequency decides the cuts, not meaning: 200 merges give un·emplo·y·ment.**

Three Ways to Learn the Pieces

scheme	what it does, and what it is after	used by
byte-pair encoding	glue the most frequent neighbouring pair: frequent strings become one token	GPT-2 to 4, LLaMA 3
WordPiece	glue the pair that most raises the training text's likelihood: the vocabulary predicts the corpus	BERT
SentencePiece unigram	start with many candidate pieces; prune those whose removal costs least likelihood	T5, XLM-R

© Zixiang Feng Redistribution prohibited without permission

- All three learn from counts on their own corpus, not from meaning: one phrase, three cuts (Sennrich et al. 2016; Schuster & Nakajima 2012; Kudo & Richardson 2018).
- **The pieces are a property of the model that learned them, not of the language.**

One Sentence, Three Tokenizers



© Zhiang Fan. Redistribution prohibited without permission

- cl100k falls back on raw computer code for a character it saw rarely (red): 央 comes out as two meaningless pieces, but the sentence still has a representation.
- English BERT knows few Chinese characters: 9 of its 12 tokens are [UNK], “unknown”, one token for every character not on its list: the sentence is lost. Chinese BERT cuts every character into its own token.
- Right column: the same two sentences in English, 10 tokens to cl100k, 20 to Chinese BERT, which spells English from fragments.
- **The same sentence is a different number of tokens to each model.**

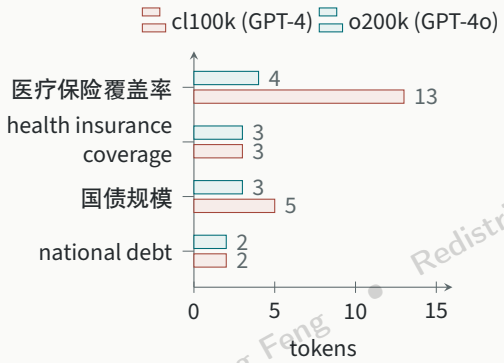
The Tokenizer Belongs to the Model



- One term, four cuts: 3 pieces, 8, 6 characters, or [UNK]s. “GDP rose by 2%”, 5 tokens cut by words, is 6 or 7 GPT tokens.
- A model’s first layer is an **embedding table**: one learned vector per token ID. Feed it another tokenizer’s IDs and every row is the wrong vector.
- Mixed Chinese–English notes and filings that mix 简 and 繁 call for byte-level BPE.
- **Report the tokenizer, and its version, as you report the sample period.**

© Zhigang Feng
Redistribution prohibited without permission

Translate, or Read in the Original?



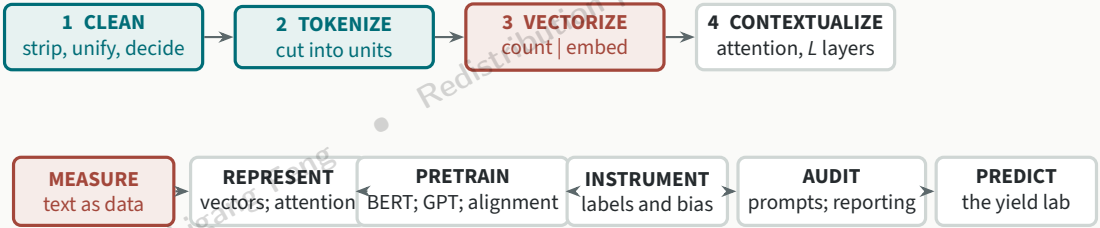
© Zhigang Feng
Redistribution prohibited without permission

- Translate a Chinese corpus and use English tools, or let a multilingual model read the Chinese?
- Translation loses shades: 稳健 is both stable and sound, “prudent” only caution; 适度宽松 is a named stance, not an adjective.
- Reading the original costs tokens; each bar is one phrase cut by one tokenizer: 医疗保险覆盖率 is 13 tokens to GPT-4’s, the English 3; GPT-4o’s cuts it to 4.
- **If a phrase is hard to translate, run both routes and report both.**

Session 5.1

Step 3 · Count

Turn tokens into vectors: one axis per word



Redistribution prohibited without permission
© Zhigang Feng

From Tokens to Numbers: IDs and One-Hot Vectors

vocabulary, numbered

	fed	cut	hike	support	curb	growth	inflation	respond
ID	1	2	3	4	5	6	7	8
$\boldsymbol{o}_{\text{fed}}$	1	0	0	0	0	0	0	0
$\boldsymbol{o}_{\text{hike}}$	0	0	1	0	0	0	0	0

$\boldsymbol{o}_{\text{fed}}^\top \boldsymbol{o}_{\text{hike}} = 0$: no two different words share a coordinate

- Number the vocabulary, $v = 1, \ldots, V$. Token v becomes the **one-hot vector** $\boldsymbol{o}_v \in \{0,1\}^V$: a 1 in place v, 0 everywhere else.
- For two different words $\boldsymbol{o}_v^\top \boldsymbol{o}_{v'} = 0$: every pair is **orthogonal** by construction. *hike* and *raise* are as far apart as *hike* and *cut*.
- **A document is the sum of its one-hot vectors: a row of counts.**

© Zhigang Feng Redistribution prohibited without permission

Bag of Words and the Document–Term Matrix



three documents, $V = 8$: $\mathbf{Z} \in \mathbb{R}^{3\times 8}$, frequency counts

- Three short documents, already cleaned and tokenized: d_1 = fed cut support growth; d_2 = fed hike curb inflation; d_3 = fed hike respond inflation inflation.
- Count each word and ignore order: the **document–term matrix** $\mathbf{Z} \in \mathbb{R}^{D\times V}$, one row per document. Binary counts record presence (inflation in d_3: 1); frequency counts, how often (2).
- **Each row is its document's vector $\mathbf{z}_i$.**

© Zhigang Feng Redistribution prohibited without permission

TF-IDF, Worked Through

term	df_v	$\mathrm{idf}_v = \log(D/\mathrm{df}_v)$	$\mathrm{tf}_{v,3}$	**TF-IDF in document 3**
fed	3	$\log(3/3) = 0$	1	0
hike	2	$\log(3/2) = 0.405$	1	0.405
inflation	2	$\log(3/2) = 0.405$	2	0.811
respond	1	$\log(3/1) = 1.099$	1	1.099

© Zixuan Feng Redistribution prohibited without permission

- $\text{TF-IDF}_{v,i} = \mathrm{tf}_{v,i} \times \log(D/\mathrm{df}_v)$: the count of word v in document i, times the natural log of how rare the word is across documents. d_3 becomes $(0, 0, 0.405, 0, 0, 0, 0.811, 1.099)$.
- A graded stop list: **template text**, wording every document repeats, such as "financial statements", has $\mathrm{df} \approx D$ and a weight near 0, with no list written in advance.
- **The trap: if every statement contains "not", its weight goes to zero too.**

The Bag Forgets Word Order

GDP growth exceeded inflation

inflation exceeded GDP growth

both become the same count vector:



the rate cut supports growth

growth supports the rate cut

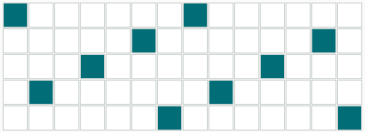
the same bag

- "GDP growth exceeded inflation" and its reverse get one vector; "the rate cut supports growth" and "growth supports the rate cut" state opposite causal directions in identical bags.
- Harmless when you count topic words, as the uncertainty index does; fatal when direction, cause or negation is the object.
- **Word order is gone. Deck 5.2 puts it back with position vectors.**

© Zhigang Feng
Redistribution prohibited without permission

Sparse: Most of the Matrix Is Zero

a document–term matrix, schematic



filled cells are non-zero; in the lab's 245 $\times$ 1,163 TF-IDF matrix, 85.5% of the cells are 0

- One corpus, three vocabularies so far, each a choice: 1,628 distinct tokens from the baseline pipeline; 1,163 TF-IDF columns (in two or more window statements, no numerals); 719 for the topic model (no stop, short or rare words).
- With fewer documents than words ($D \ll V$), OLS on all V columns is not identified: $\mathbf{Z}^\top\mathbf{Z}$ cannot be inverted. Reduce the dimension (topic models) or penalise (the Lasso).
- **That is why text regressions borrow the tools of machine learning.**

© Zhigang Feng
Redistribution prohibited without permission

Comparing Documents: Cosine on Counts

	fed	cut	hike	support	curb	growth	inflation	respond	raise
d_1	1	1	0	1	0	1	0	0	0
d_2	1	0	1	0	1	0	1	0	0
d_3	1	0	1	0	0	0	2	1	0
d_2'	1	0	0	0	1	0	1	0	1

$d_2 \cdot d_3 = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 2 = 4$
$\|d_2\| = \sqrt{1+1+1+1} = 2$, $\|d_3\| = \sqrt{1+1+2^2+1} = \sqrt{7}$
$\cos(d_2, d_3) = 4/(2\sqrt{7}) = 0.756$; $\cos(d_1, d_2) = 1/(2 \cdot 2) = 0.25$
$\cos(d_2', d_3) = (1+2)/(2\sqrt{7}) = 0.567$, d_2' is d_2 with *raise* for *hike*

- Words are orthogonal; documents are not. A row is a sum of one-hots, so in the **cosine similarity** $\mathbf{z}_i^\top \mathbf{z}_j / (\|\mathbf{z}_i\| \|\mathbf{z}_j\|)$ every word two documents share adds to the numerator.
- d_1, d_2 share only *fed*: $1/(2 \cdot 2) = 0.25$; d_2, d_3 share three words: 0.756.
- Swap *hike* for *raise* in d_2: same meaning, but the cosine with d_3 falls from 0.756 to 0.567.
- **Counts see shared strings, not shared meaning; Deck 5.2 repairs this.**

© Zhaogang Fan 2026 Redistribution prohibited without permission

Three Weightings of the Same Two Documents

d_2 = fed hike curb inflation
d_3 = fed hike respond inflation inflation

	fed	cut	hike	support	curb	growth	inflation	respond
counts d_2	1	0	1	0	1	0	1	0
d_3	1	0	1	0	0	0	2	1
presence d_2	1	0	1	0	1	0	1	0
d_3	1	0	1	0	0	0	1	1
TF-IDF d_2	0	0	.41	0	1.10	0	.41	0
d_3	0	0	.41	0	0	0	.81	1.10

$\cos(d_2, d_3)$: counts $4/(2\sqrt{7}) = 0.756$; presence $3/(2 \cdot 2) = 0.75$; TF-IDF $0.49/(1.24 \cdot 1.42) = 0.28$

- **Counts**: $z_{iv} = n_{iv}$. **Presence**: $z_{iv} = \mathbf{1}[n_{iv} > 0]$. **TF-IDF**: $z_{iv} = n_{iv}\,\mathrm{idf}_v$, $\mathrm{idf}_v = \log(D/\mathrm{df}_v)$ from the fitting documents.
- Three questions for a cosine: same words in the same amounts? same words at all? same *unusual* words?
- Template text (here *fed*) counts in full, once, or not at all: d_1, d_2, sharing only *fed*, score 0.25, 0.25, 0.
- **Same documents, three cosines: every cosine inherits the choice.**

© Zhiyang Feng • Redistribution prohibited without permission

The Data: 245 FOMC Statements, One Template Edited Each Meeting

29 Oct. 2008

… decided today to lower its target for the federal funds rate 50 basis points to 1 percent.

In light of the declines in the prices of energy and other commodities and the weaker prospects for economic activity, the Committee expects inflation to moderate in coming quarters to levels consistent with price stability.

16 Dec. 2008

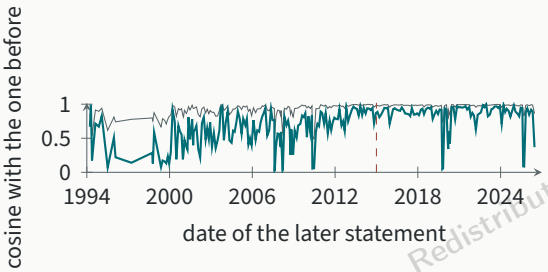
… decided today to establish a target range for the federal funds rate of 0 to 1/4 percent.

In light of the declines in the prices of energy and other commodities and the weaker prospects for economic activity, the Committee expects inflation to moderate further in coming quarters.

© Zhirang Feng · Redistribution prohibited without permission

- The corpus: every FOMC post-meeting statement, 1994-02-04 to 2026-06-17 (245 texts). Each is drafted from the one before: grey carried over word for word, red changed.
- Next: how much did the message change meeting to meeting? The cosine with the statement before, under each weighting; 10 Aug. 2007, a liquidity notice with no decision, is the odd text.
- **Carried-over template text: counts weigh it in full, TF-IDF near zero.**

Cosine on Real Statements: Three Weightings, Three Answers



pair	counts	0/1	TF-IDF
mean, 244 pairs	0.93	0.80	0.70
share above 0.9	79%	39%	28%
Oct. → Dec. 2008	0.81	0.55	0.26
7 → 10 Aug. 2007	0.56	0.26	0.02

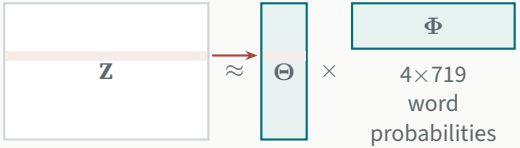
dashed line: vocabulary and IDF fitted on the statements left of it

Redistribution prohibited without permission

- x: the later date; y: cosine with the statement before; grey counts, teal TF-IDF.
- Counts: a typical pair scores 0.93, 79% lie above 0.9, Dec. 2008's rewrite 0.81: the template dominates. TF-IDF discounts it: mean 0.70, 28% above 0.9, and its dips are the meetings whose news changed (Dec. 2008 0.26; the Aug. 2007 notice 0.02).
- **Choose the weighting by the question: "same wording?" is counts, "same news?" is TF-IDF.**

© Thijs Knaap

Topic Models: What We Have and What We Want

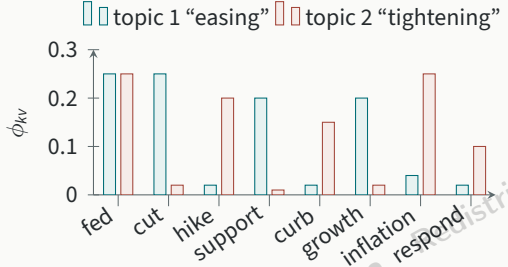


the red row: one statement, 719 counts on the left, 4 topic shares on the right

- What we have: **Z**, one row of 719 counts per statement: too many to regress on. LDA compresses them, as a factor model does.
- What we want: $K = 4$ topics, each a list of word probabilities ϕ_{kv} (what the topic says), and per statement K shares θ_{ik} (how much of each it uses).
- How: assume a story (next frame); guess each word's topic, improve the guesses, count.
- **Text enters a regression as K shares, not V counts.**

© Zhigang Feng Redistribution prohibited without permission

The Assumption: How a Document Is Written, Word by Word



d_3 with $\boldsymbol{\theta}_3 = (0.1,\ 0.9)$, one draw:

hike	inflation	respond	inflation	fed
$z = 2$	$z = 2$	$z = 2$	$z = 2$	$z = 1$

- **Latent Dirichlet allocation** (Blei, Ng & Jordan 2003) assumes a story. Top: topic k's **distribution over words**, its bars ϕ_k.
- Two kinds of dice: each document its own K-sided die, its **topic shares** θ_i (d_3: 0.1, 0.9); each topic one V-sided die, its bars ϕ_k, shared by all. Per slot: roll the document's die for a topic z, then die z for a word w.
- **A document mixes topics; word order plays no part.**

© Zhiguo Feng · Redistribution prohibited without permission

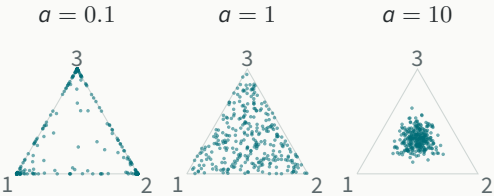
LDA's Symbols: Two Unknowns, Two Count Tables

symbol	what it is	size	toy	FOMC fit
z	the **tag** on one token: the topic that produced it	1 per token	*fed* in d_3: $z = 1$	21,673 tags
ϕ_k	the **bars**: topic k's distribution over words (the chance it produces each word); sums to 1	V per topic	the two bar charts	4×719
θ_i	the **shares**: document i's distribution over topics (the fraction of its words from each); sums to 1	K per document	$\theta_3 = (0.1,\ 0.9)$	149×4
$n_k^{(v)}$	the **word table**: tokens of word v now tagged k; a row, normalised, is $\hat{\phi}_k$	$K \times V$ counts	worked frame, top	4×719
$n_i^{(k)}$	the **document table**: tokens of document i now tagged k; a row, normalised, is $\hat{\theta}_i$	$D \times K$ counts	worked frame, bottom	149×4
$a,\ b$	pseudo-counts added before normalising, so no share or bar is ever 0	two numbers	$0.1,\ 0.01$	$0.1,\ 0.01$

© Zhigeng Geng. Redistribution prohibited without permission

- Fitting runs the story backward: words $\rightarrow$ every tag $z \rightarrow \hat{\phi}, \hat{\theta}$ read off the two tables.
- **Bars and shares are the unknowns; the two tables are the counts behind them.**

Why “Dirichlet”: A Knob on How Focused a Document Is



$K = 3$; a corner = all words from that topic; a point = one document’s shares θ; 300 draws each. $a = 0.1$: 67% of draws give one topic more than 0.9; $a = 10$: in 94% every share is within 0.2 of $1/3$.

- The **Dirichlet** is a distribution over shares: each draw is a whole die. LDA draws each θ_i from $\mathrm{Dir}(a)$, each ϕ_k from $\mathrm{Dir}(b)$.
- Small a: one or two topics per document; large a: all of them (left). b: the same for a topic’s words.
- The same a as in the read-off, $\hat{\theta}_{ik} = (n_i^{(k)} + a)/(n_i + Ka)$: a pretend count added to every topic.
- **FOMC fit: $a = 0.1$, $b = 0.01$: few topics per statement, few words per topic.**

© Zhigang Feng — Redistribution prohibited without permission

Fitting LDA: The Question and the Idea

a current guess: every token carries a topic tag

d_1 fed cut support growth

d_2 fed hike curb inflation

d_3 fed hike respond inflation inflation

teal: topic 1; red: topic 2; white: the token re-guessed. Were these tags right, fitting is counting (next frame): 8 red tokens, 3 of them *inflation*: $\hat{\phi}_{2,\text{inflation}} = 3/8$; d_3's other tags all red: $\hat{\theta}_3 \approx (0, 1)$.

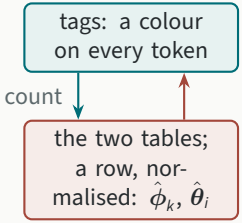
which topic likes *fed*?

which topic does d_3 use?

- We see only words and want each token's tag z; with the tags known, the bars $\hat{\phi}$ and shares $\hat{\theta}$ are the frequencies on the left.
- Circular: tags need ϕ, θ, which need tags. The way out: hold every other tag fixed and re-guess one, from the two questions below left.
- Token by token, round after round, the guesses settle on the posterior (**Gibbs sampling**).
- **"Collapsed": ϕ, θ are never drawn, only counted.**

© Zingang Feng Unauthorized redistribution prohibited without permission

The Loop: Tags Give Counts, Counts Judge Tags



down, as drawn on the previous frame: 8 red tokens, 3 of them *inflation*: $\hat{\phi}_{2,\text{inflation}} = 3/8$; d_3 all red: $\hat{\theta}_3 = (0, 1)$. Up, *fed* in d_3 drawn teal (1 in 20): $n_1^{(\text{fed})}$ $1 \to 2$, $n_1^{(\cdot)}$ $4 \to 5$, $n_3^{(1)}$ $0 \to 1$, so $\hat{\phi}_{1,\text{fed}} = 1/4 \to 2/5$, $\hat{\theta}_3 = (0, 5)/5 \to (1, 4)/5$; drawn red: nothing moves (*a*, *b* left out here).

- Down: with the tags in place, bars and shares are the tables' rows, normalised; nothing else is estimated.
- Up: no answer key exists. A tag is judged only by the other tags, through the tables: does topic *k*'s row favour this word, does this document's row favour *k*?
- A re-drawn tag moves one count in each table, and bars and shares move with it (left); they are never set by hand.
- **Stop when the tags agree with each other; the thermometer says when.**

© Yinzhu Feng • Redistribution prohibited without permission

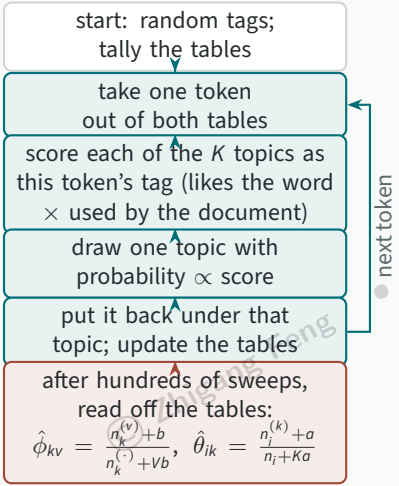
Gibbs Sampling in Words: Labels That Correct Each Other



© Zhaoyang Feng · Redistribution prohibited without permission

- Gibbs sampling turns words into tags; $\hat{\phi}$, $\hat{\theta}$ are just the tags counted (below each panel).
- One token at a time: hide its colour; ask which colour this word carries elsewhere and which this document's other words carry; multiply, roll a weighted die, colour it.
- Why it settles: words that keep appearing together lift each other's chance of one colour, in and across documents; a lone mistake has no support and is overwritten.
- The log-likelihood is a thermometer, not the engine: when it stops rising, stop and count.
- **No one labels by hand: each label is re-guessed from all the others until they agree.**

Collapsed Gibbs Sampling: Start, One Step, Stop



1. **Start.** Random tags; tally the two count tables.
2. **One step**, for word v in document i, tagged k: subtract 1 from $n_k^{(v)}$, $n_k^{(\cdot)}$, $n_i^{(k)}$ (its own tag must not be evidence); for every topic, s_k = how plausible it is that topic k produced this token:

$$s_k = \underbrace{\frac{n_k^{(v)} + b}{n_k^{(\cdot)} + Vb}}_{\hat{\phi}_{kv}\text{: topic } k\text{'s share of } v} \underbrace{\left(n_i^{(k)} + a\right)}_{\propto\, \hat{\theta}_{ik}\text{: } i\text{'s tokens tagged } k}$$

   $p_k = s_k / \sum_j s_j$; draw $u \sim U[0,1]$, the first k with $p_1 + \cdots + p_k > u$ wins; add 1 to the same three counts for that k.
3. **Stop.** A **sweep** visits every token once; run hundreds; read off (red box).

Redistribution prohibited without permission
Zhaoguang Wang

One Step, Worked: the Toy's Two Tables, Then *fed* in d_3

word table, all three documents pooled: row = topic, column = word, cell = count

	fed	cut	hike	support	curb	growth	inflation	respond	
$k=1$	1	1	0	1	0	1	0	0	$n_1^{(\cdot)}=4$
$k=2$	2	0	2	0	1	0	3	1	$n_2^{(\cdot)}=9$

document table: tokens tagged 1 (teal) and 2 (red), per document

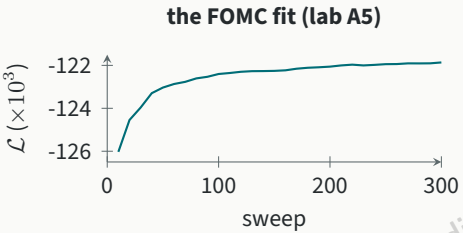
d_1	4	0	d_2	0	4	d_3	0	5

after the draw, the counts that change: red (19 in 20): back to $n_2^{(\text{fed})}=2, n_2^{(\cdot)}=9$, $n_3^{(2)}=5$; teal (1 in 20): $n_1^{(\text{fed})}=2, n_1^{(\cdot)}=5$, $n_3^{(1)}=1$, topic 2 keeps 1, 8, 4

- **Out.** *fed* in d_3 is red; subtract 1: $n_2^{(\text{fed})}$ $2\to1$, $n_2^{(\cdot)}$ $9\to8$, $n_3^{(2)}$ $5\to4$. Every other tag, *fed* in d_1 and d_2 included, stays and votes.
- **Scores** ($a=0.1$, $b=0.01$ our choice; $V=8$): $s_1=\frac{1.01}{4.08}\times0.1=0.025$ (likes *fed*, 1 of 4; d_3 never uses it); $s_2=\frac{1.01}{8.08}\times4.1=0.513$ (1 of 8; d_3 uses it four times). $p=(0.05,\ 0.95)$.
- **Draw, put back.** Red, 19 times in 20: add 1 back, the counts return to 2, 9, 5. Teal, once in 20: three topic-1 counts rise instead (left); the next token sees the new tables.
- **Draw; never take the largest p_k: that one chance in twenty is how the chain explores.**

© Zhiqiu Feng Redistribution prohibited without permission

The Thermometer: How Probable Are the Words Under the Current Fit?



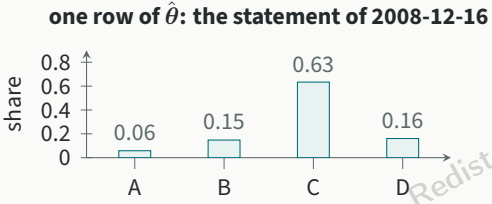
x = sweep; $y = \mathcal{L}$ computed from the tables after that sweep: it rises from -126,029 and is flat by 300 sweeps (-121,864).

$$\mathcal{L} = \sum_{\text{documents } i} \sum_{\text{tokens } v \text{ of } i} \log \underbrace{\sum_k \hat{\theta}_{ik}\, \hat{\phi}_{kv}}_{\text{chance of this word in this document}}$$

- Inside the log: $\sum_k \hat{\theta}_{ik}\hat{\phi}_{kv}$, the products one step scores with; p_k is topic k's share of the sum. Log; add over every token.
- Nonsense tags: every topic a blur of all words, every document of all topics; each observed word improbable: $\mathcal{L}$ low.
- Tags sorting themselves: topics concentrate on their words, documents on theirs; words more probable: $\mathcal{L}$ rises, then flat: stop.
- **We never climb it; we read it.**

© Zhigang Feng — Redistribution prohibited without permission

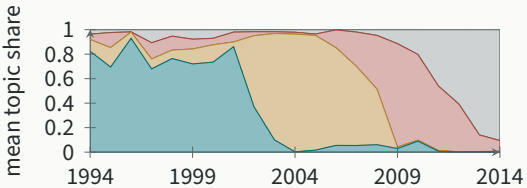
Lab A5: Topics in the FOMC Statements, the Setup



- Can 4 shares stand in for 719 counts in the 1994–2014 statements?
- The thermometer (previous frame) is flat by 300 sweeps: the chain has settled, so we read the tables.
- Left: $\hat{\theta}$ of the 2008-12-16 statement: x = topic, y = share of its tokens; 0.63 in C.
- **The fit:** $K = 4$**,** $a = 0.1$**,** $b = 0.01$**, 300 sweeps.**

© Zhigang Feng
Redistribution prohibited without permission

LDA on the FOMC Statements: What It Found

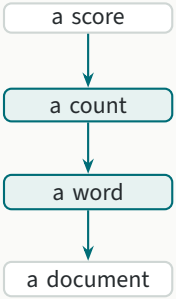


top words, and the name we gave
reserve, discount, board, available *A: discount-rate announcements*
chairman, risks, decided, voting *B: the vote and the balance of risks*
financial, markets, levels, housing *C: crisis, markets and housing*
longer, term, consistent, pace *D: longer-run goals and purchases*

© Zhirang Feng · Redistribution prohibited without permission

- Area: x = year, y = mean $\hat{\theta}$ over the year's statements, bands stacking to 1. Table: each topic's four largest $\hat{\phi}_{kv}$. Topics come back numbered, not named: the names are ours.
- Four drafting eras (A to 2002, B from the 2002 vote roster, C 2009–11, D 2012–14): the word clusters track how the statement was written more than what the economy did.
- **Read a topic's words and its time path before its share enters a regression.**

When Counting Still Wins

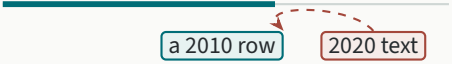


every coordinate is a word, so a score can be traced back to its source

- When the question is how often defined words occur, transparency is the value: the uncertainty index, finance sentiment lists, political risk.
- An audit traces a score back to its words and documents; a dense vector cannot.
- Cosines of binary word vectors of 10-K product descriptions define product-market rivals (Hoberg & Phillips 2016); the same can be tried on the business sections of other annual reports.
- **Under about a thousand documents, TF-IDF is the sound baseline.**

© Zhigang Feng
Redistribution prohibited without permission

No Look-Ahead: Fit the Weights on Past Statements Only

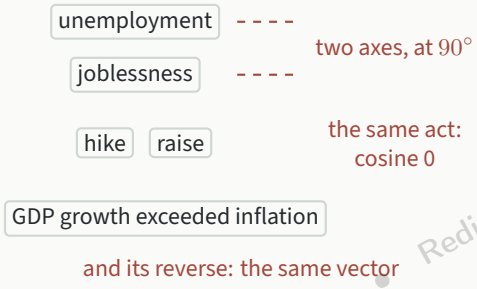


	fit on 1994–2014		fit on all 245	
word	df	IDF	df	IDF
balance	33	1.51	89	1.01
securities	49	1.11	120	0.71
pandemic	0	no column	19	2.56

- Which documents does df_v in $\log(D/\mathrm{df}_v)$ count? Left: two answers.
- Over all 245: *balance* is in 56 of the 96 later statements, so a 2010 row weighs it 1.01, not 1.51: a 2010 number built from 2020 texts.
- **Look-ahead**: a 2010 forecast built on it (Deck 5.6) uses texts no one had in 2010. The rule: fit on 1994–2014, apply unchanged; *pandemic* gets no column.
- **"Fitted on the future": a df that counts documents dated after the row.**

© Zhigang Feng
Redistribution prohibited without permission

What Counts Cannot See



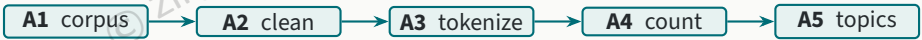
- Cleaning, tokenizing and counting turned each document into a row of counts: transparent, auditable, and blind to meaning.
- Synonyms sit on perpendicular axes, word order is gone, and every word is its own axis.
- Deck 5.2 gives up one axis per word: words become points, and nearness means similarity.
- **Counts tell you which words were used, not what was meant.**

© Zhigang Feng
Redistribution prohibited without permission

Lab, Part A: Clean, Tokenize and Count the FOMC Statements

cell	what you run	the question you answer
A1	load 245 statements; set the 1994–2014 window	How many statements may a fitted object see?
A2	five cleaning switches, one at a time	Which switch moves the vocabulary most, and what does it delete?
A3	byte-pair encoding, 200 merges	How is a word the window never saw, such as *reflationary*, cut?
A4	document–term matrix and TF-IDF, fitted on the window	How much held-back text falls outside the vocabulary? What would a full-sample IDF change?
A5	LDA with $K = 4$, collapsed Gibbs	Your names for the topics, a neighbour's, and what changes with seed 7 or $K = 6$?

© Zhigang Feng · Redistribution prohibited without permission



- **Every number on the counting slides is one you can reproduce.**

Session 5.2

Meaning in Vectors: Embeddings, Attention, the Transformer

Embed, then contextualize: from one vector per word to one vector per word in its sentence

Based on Chapter 13 of Miao & Feng, *AI for Economists*

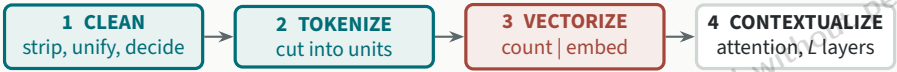
Redistribution prohibited without permission



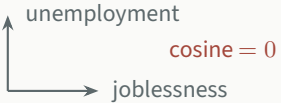
© Zhigang Feng

Zhigang Feng

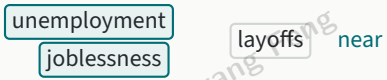
Where the Pipeline Stands



counting: one axis per word



embedding: one point per word



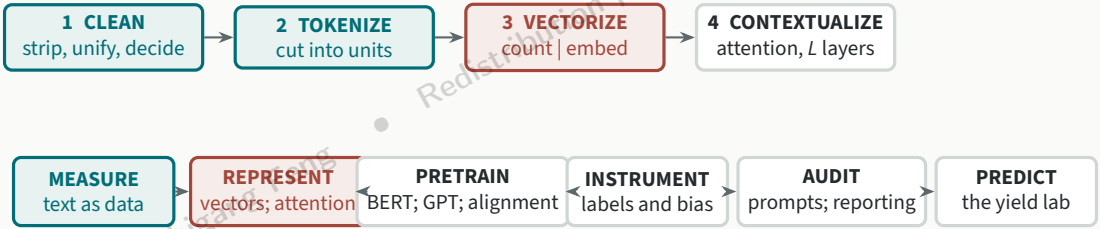
- Deck 5.1 cleaned, tokenized and counted: each document became a **vector**, one entry per vocabulary word.
- Every word is its own axis, so two different words have **cosine** (of their angle) 0: *unemployment* and *joblessness*, *hike* and *raise*.
- **The repair: make words points, where nearness means similar meaning.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

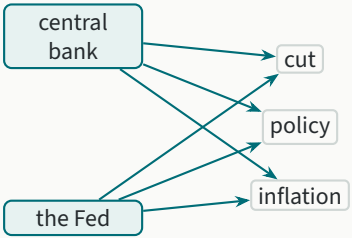
Step 3 · Embed

Words become points, and closeness becomes meaning



© Zhigang Feng
Redistribution prohibited without permission

Know a Word by the Company It Keeps



two words, one set of neighbours: their vectors land close

- Firth (1957): "You shall know a word by the company it keeps," the **distributional hypothesis**.
- *Central bank* and *Fed* both appear near *cut*, *policy* and *inflation*. Nobody declared them related; their contexts did.
- A **distributed representation**: one dense vector $\boldsymbol{u}_w \in \mathbb{R}^d$ per word, $d \ll V$, learned from co-occurrence. Its axes carry no names.
- **Similar context profiles, nearby vectors: the orthogonality of counting is gone.**

© Zhigang Feng Redistribution prohibited without permission

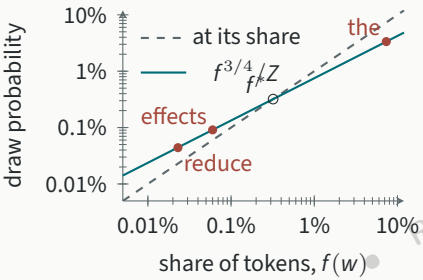
Word2Vec, Step 1: A Window Turns Text Into Pairs



- Each token in turn is the **centre word**; the Δ tokens on each side of it (here $\Delta = 2$) are its **context words**. Each (centre, context) pair is a **real pair**: label 1.
- For every real pair, K **noise words** are drawn at random from the vocabulary and paired with the same centre: label 0. The next frame says how they are drawn.
- **The corpus supplies the data: real pairs against random ones (Mikolov et al. 2013).**

© Zhigang Feng redistribution prohibited without permission

How the Noise Words Are Drawn



- $P_{\text{noise}}(w) = f(w)^{3/4}/Z$ with $Z = \sum_w f(w)^{3/4} = 4.21$ here; $f(w)$: w's share of the 624-word vocabulary's tokens.
- Why not $\propto f$: the noise column fills with *the*, *of*, *to*. Uniform: rare words swamp it.
- $f^{3/4}$ alone never dips below the line; $Z > 1$ pulls it down. Above $f^* = Z^{-4} = 0.32\,\%$ of tokens a word is drawn below its share (*the*: 0.46×), below it above (*reduce*: 1.93×).
- **Negative examples cost nothing: drawn, not labelled. $K = 2$ here, 5–20 in practice.**

© Zhigang Feng
Redistribution prohibited without permission

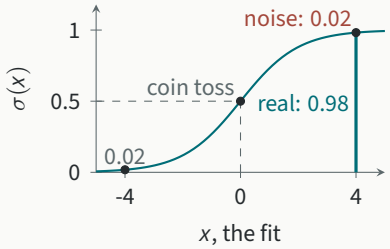
Step 1, Worked: One Centre Word Becomes 12 Rows

real pair	label	its $K = 2$ noise pairs, drawn at random	label
(*raised*, *central*)	1	(*raised*, *next*), (*raised*, *low*)	0
(*raised*, *bank*)	1	(*raised*, *lending*), (*raised*, *3*)	0
(*raised*, *interest*)	1	(*raised*, *in*), (*raised*, *less*)	0
(*raised*, *rates*)	1	(*raised*, *susan*), (*raised*, *earlier*)	0

© Hugo Freund. Redistribution prohibited without permission

- Where we are: the corpus has no labels, so Step 1 makes them. The chapter's sentence, centre *raised*, $\Delta = 2$: four real pairs, label 1 (left column).
- Right column: for each real pair, $K = 2$ words drawn from the FOMC vocabulary by the last frame's rule (seed 42), label 0. Nobody checks them against the text.
- 12 rows from this centre word; then *interest* is the centre, 12 more, and so on through all T tokens: about $T \cdot 2\Delta(1 + K)$ rows, every label free.
- **Next: a classifier that tells the 1s from the 0s, with the word vectors as its only parameters (two frames), fitted one block at a time (the frame after).**

Word2Vec, Step 2: From a Row to a Probability



- A row is a pair (w, c): a centre and a word offered as its context. Its **fit** $x = \tilde{\boldsymbol{u}}_c^\top \boldsymbol{u}_w$ is large when the two vectors point the same way ($\boldsymbol{u}_w$ as centre, $\tilde{\boldsymbol{u}}_w$ as context: $2Vd$ parameters; $\boldsymbol{u}_w$ is kept).
- $\sigma(x) = 1/(1 + e^{-x})$ turns the fit into the chance the pair is real, $P(\text{label } 1) = \sigma(x)$. Noise is the only other label: $P(\text{label } 0) = 1 - \sigma(x)$, the gap up to 1, by symmetry $\sigma(-x)$.
- Read the plot: $x = 0$, a coin toss; $x = 4$, real with chance 0.98, noise 0.02; $x = -4$ the reverse.
- **A binary logit on Step 1's rows; the vectors are its only coefficients.**

© Zhigang Feng
Redistribution prohibited without permission

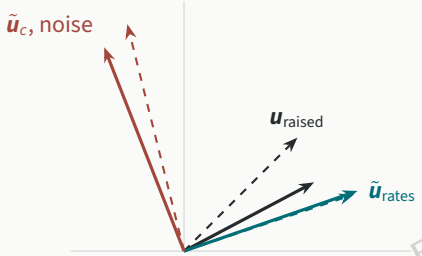
Word2Vec, Step 2: The Objective, Term by Term

$$\max_{\{\boldsymbol{u},\tilde{\boldsymbol{u}}\}} \sum_{t=1}^{T} \sum_{j\neq 0,\ |j|\leq\Delta} \Bigg[\underbrace{\log\sigma\big(\tilde{\boldsymbol{u}}_{w_{t+j}}^{\top}\boldsymbol{u}_{w_t}\big)}_{\text{the real pair: } \log P(\text{label } 1)} + \underbrace{\sum_{k=1}^{K}\log\sigma\big(-\tilde{\boldsymbol{u}}_{c_k}^{\top}\boldsymbol{u}_{w_t}\big)}_{\text{its } K \text{ noise pairs: } \log P(\text{label } 0)} \Bigg]$$

- Inside the brackets, one block of the table: the real pair's log-probability of being real plus its K noise pairs' log-probability of being noise, $c_k \sim P_{\text{noise}}$.
- The real term rises with $x^{+} = \tilde{\boldsymbol{u}}_{w_{t+j}}^{\top}\boldsymbol{u}_{w_t}$: the context's vector is pulled towards the centre's. A noise term rises as $x^{-} = \tilde{\boldsymbol{u}}_{c_k}^{\top}\boldsymbol{u}_{w_t}$ falls: a random word's is pushed away.
- Why both: with real pairs alone, all vectors equal and long would make every $\sigma \to 1$; the noise rows forbid it: near its company, away from random words.
- Outside: t runs over every token as the centre, j over the 2Δ window slots, so over every block the corpus yields; the sum is the log-likelihood of all the labels.
- **Same company, same pulls, same vector: the next frame takes one step.**

© Edgar Fuller • Redistribution prohibited without permission

One Step: Pull the Real Context Closer, Push the Noise Away



dashed: before; solid: after one step (schematic)

- A real pair (raised, rates), $x^+ = \tilde{u}_{\text{rates}}^\top u_{\text{raised}}$; a noise pair (raised, c), $x^- = \tilde{u}_c^\top u_{\text{raised}}$. One step moves u_{raised} by $\eta[(1-\sigma(x^+))\,\tilde{u}_{\text{rates}} - \sigma(x^-)\,\tilde{u}_c]$.
- The real context pulls, hardest while the model doubts the pair ($\sigma(x^+)$ small); the noise pushes, hardest when the model believes it ($\sigma(x^-)$ large).
- The context vectors move by the mirror terms: $\tilde{u}_{\text{rates}}$ towards u_{raised}, $\tilde{u}_c$ away.
- **Same contexts, same pulls: such words end up pointing the same way.**

© Zhigang Feng Redistribution prohibited without permission

Skip-Gram, CBOW and GloVe in Plain Words

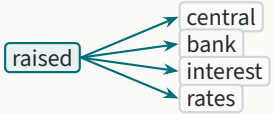


© Zhang Yue Redistribution prohibited without permission

- Skip-gram: show one word, hide its neighbours, let the vectors guess them; a wrong guess nudges the vectors. One lesson per neighbour, so a rare word still gets its own.
- CBOW: show the neighbours, hide the centre word, average their vectors and guess it. One lesson per window: faster, but a rare word's lesson is diluted in the average.
- GloVe: no guessing. Count once how often each pair shares a window, then choose vectors whose dot product matches the log count, frequent pairs weighted more.
- **Three routes, one end: same company, same direction.**

Skip-Gram and CBOW: Same Window, Different Regressor

skip-gram: the centre predicts each context word



CBOW: the window's mean predicts the centre



skip-gram, one row per pair (t, j): $\log \sigma(\tilde{\boldsymbol{u}}_{w_{t+j}}^\top \boldsymbol{u}_{w_t}) + \sum_k \log \sigma(-\tilde{\boldsymbol{u}}_{c_k}^\top \boldsymbol{u}_{w_t})$

CBOW, one row per window t, $\bar{\boldsymbol{h}}_t = \frac{1}{2\Delta} \sum_{j \neq 0} \boldsymbol{u}_{w_{t+j}}$: $\log \sigma(\tilde{\boldsymbol{u}}_{w_t}^\top \bar{\boldsymbol{h}}_t) + \sum_k \log \sigma(-\tilde{\boldsymbol{u}}_{c_k}^\top \bar{\boldsymbol{h}}_t)$

	skip-gram	**CBOW**
predicting side	the centre word, $\boldsymbol{u}_{w_t}$	the window's mean, $\bar{\boldsymbol{h}}_t$
predicted side	each context word, $\tilde{\boldsymbol{u}}_{w_{t+j}}$	the centre word $\tilde{\boldsymbol{u}}_{w_t}$, or a noise word
rows per window	2Δ real $+ 2\Delta K$ noise	1 real $+ K$ noise: 2Δ times fewer, so faster
one step moves	the pair's two vectors, in full	one step, spread over the 2Δ context words

- **Same σ, same logit, same window; only what predicts differs: one word or the window's mean. Skip-gram: a rare word's every occurrence is its own row.**

© Zhigang Feng Redistribution prohibited without permission

GloVe: Count First, Then Fit the Table

$$\log N_{vv'} \approx b_v + \tilde{b}_{v'} + \boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$$

	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$	$\tilde{b}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$
b_v	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$	$\boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'}$

one cell per word pair (Pennington et al. 2014)

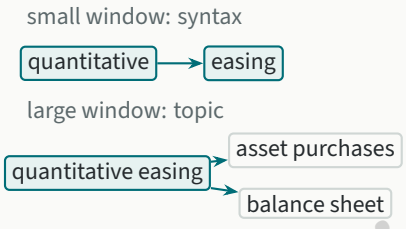
- What we have: the $V \times V$ table $N_{vv'}$, how often v' falls within v's window, the one B1 builds. Word2vec never writes it down; GloVe fits it.
- A gravity equation, trade = size × size × closeness: b_v, $\tilde{b}_{v'}$ absorb each word's frequency, the dot product how much more than chance the two meet (B1's PMI).

$$\min_{\boldsymbol{u},\tilde{\boldsymbol{u}},b,\tilde{b}} \sum_{v,v'} h(N_{vv'}) \left(b_v + \tilde{b}_{v'} + \boldsymbol{u}_v^\top \tilde{\boldsymbol{u}}_{v'} - \log N_{vv'}\right)^2$$

- Weighted least squares over the cells: $h(0) = 0$ drops the empty cells, and h is capped, so the commonest pairs do not dominate (the chapter's two roles of h).
- **Three routes to one object: word2vec streams the rows, GloVe fits the count table, B1 factorises it by SVD; all reward words that keep the same company.**

© Zhiyuan Feng. Redistribution prohibited without permission

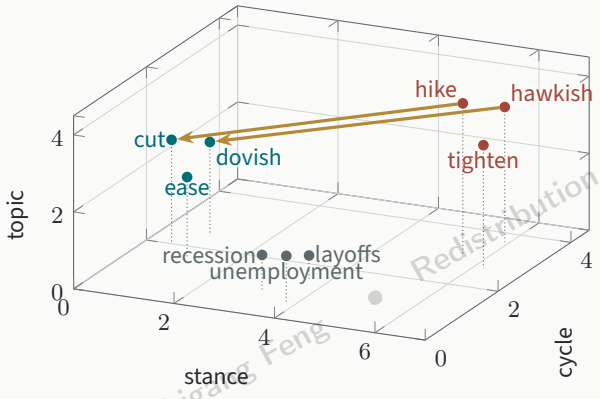
Window, Dimension, Domain



- Small windows ($\Delta \leq 3$) find syntax; large ones ($\Delta \geq 10$) find topic, usually what a measure wants. The vector length *d* is our choice too, usually 100–300.
- Domain matters more: trained on general text, *discount* sits near shop markdowns, not the discount rate; *yield* near harvests, not bond yields.
- **Train on the corpus you measure.**

© Zhigang Feng Redistribution prohibited without permission

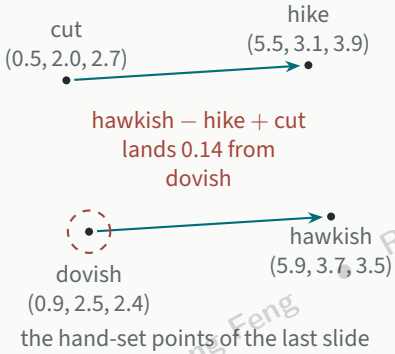
The Geometry: Clusters, Parallel Moves, Coupled Axes



© Zhigang Feng
Redistribution prohibited without permission

- Nine points, hand-set on named axes; real vectors have 100–300 unnamed ones.
- **Clusters**: tightening (red), easing (teal), the cycle (grey).
- **Parallel moves**: hike→cut and hawkish→dovish point the same way (gold).
- **Coupling**: *hike* is high on stance *and* topic.
- **A direction can mean something; an axis rarely does.**

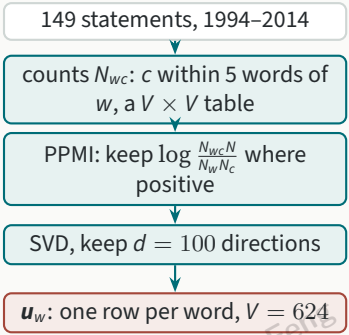
Analogies, Worked, and Why the Metric Is Cosine



© Zhigang Feng
Redistribution prohibited without permission

- **Parallelogram rule:** $\boldsymbol{u}_A - \boldsymbol{u}_B \approx \boldsymbol{u}_C - \boldsymbol{u}_D$. Here hike − cut = (5.0, 1.1, 1.2) and hawkish − dovish = (5.0, 1.2, 1.1): cosine 0.9996.
- hawkish − hike + cut = (0.9, 2.6, 2.3), 0.14 from dovish. The famous case: king − man + woman ≈ queen.
- $\|\boldsymbol{u} - \boldsymbol{u}'\|^2 = \|\boldsymbol{u}\|^2 + \|\boldsymbol{u}'\|^2 - 2\|\boldsymbol{u}\|\|\boldsymbol{u}'\| \cos\theta$: distance also reads the lengths, and length tracks frequency.
- **Compare directions with cosine, not lengths with distance.**

Lab B1: Word Vectors From the FOMC Statements, the Setup



- The question: what do the Committee's own words mean, judged by their company in its statements? Vectors trained on news would answer for news.
- 149 statements are too few for skip-gram, so lab cell B1 counts, on words seen $\geq$10 times.
- PMI: how much more often w and c meet than chance (N: all pairs). Then $\mathrm{PPMI} \approx \mathbf{U}_d \boldsymbol{\Sigma}_d \mathbf{V}_d^\top$; $\boldsymbol{u}_w$ = row w of $\mathbf{U}_d \boldsymbol{\Sigma}_d^{1/2}$, length 1.
- **A close cousin of SGNS, whose optimum has $\tilde{\boldsymbol{u}}_c^\top \boldsymbol{u}_w = \mathrm{PMI}(w, c) - \log K$ (Levy & Goldberg 2014).**

© Zhigang Feng · Redistribution prohibited without permission

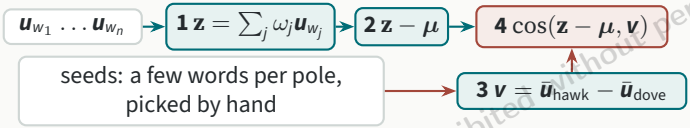
What the FOMC Vectors Learned

word	its five nearest neighbours by cosine				
inflation	expectations	longer	pressures	term	stable
employment	maximum	objectives	seeks	foster	toward
committee	today	decided	open	raise	particular
accommodation	remove	can	removed	begin	take

© Zhiwei Feng. Unauthorized distribution prohibited without permission

- Read the neighbours as phrases: *inflation* meets *expectations*, *longer* and *pressures* in "longer-term inflation expectations" and "inflation pressures"; *maximum employment* is the mandate's. A 5-word window learns set phrases.
- "raise : raised :: lower : ?" has no answer: *raised* occurs fewer than 10 times and gets no vector — the method, not the geometry.
- In B1: print other words' neighbours; change the window or minimum count; see what survives.
- **Read the neighbours before trusting a direction.**

From Word Vectors to a Document Score: The Recipe



© Zbigniew Pastuszak. Free of redistribution prohibited without permission

- Seeds, the only hand input: a few words that mean one pole and nothing else, picked from the statements or a published list, and reported. Here *hike, tighten, hawkish* / *cut, ease, dovish*; B2: *raise, increase, increased, increasing, tightening, firming* / *lower, easing, eased, accommodative, accommodation*.
- **1** $\mathbf{z}$: the document's word vectors averaged, weights ω_j preset ($1/n$; TF-IDF in B2).
- **2** $\boldsymbol{\mu} = \frac{1}{V}\sum_w \boldsymbol{u}_w$, the average word; $\mathbf{z} - \boldsymbol{\mu}$ is what the document says beyond it.
- **3** $\boldsymbol{v}$: the hawkish seeds' mean minus the dovish seeds' mean, pole to pole. **4** score $= \cos(\mathbf{z} - \boldsymbol{\mu}, \boldsymbol{v})$: $+1$ hawkish, -1 dovish, 0 neither.
- **Seeds, weights and centring are three choices, and the score is a regressor: validate it like one (Gennaro & Ash 2022; Ash & Hansen 2023).**

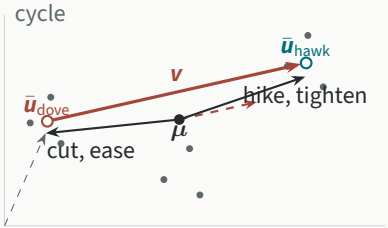
Reading the Picture: Words Are Points, a Document Is a Point, Stance Is an Arrow



- Every word is a point; a document is the average of its words' points, one point $\mathbf{z}$. The average of all words, $\boldsymbol{\mu}$, is the centre of the cloud.
- Two hand-picked lists give two poles: the hawkish words' centre and the dovish words' centre. The arrow from one to the other is the stance direction $\boldsymbol{v}$.
- Score $\mathbf{z}$: draw its arrow from $\boldsymbol{\mu}$; $+1$ if it runs along $\boldsymbol{v}$ (hawkish), -1 against (dovish), 0 at right angles (something else).
- **Measure from the centre, along the pole-to-pole arrow.**

© Zhigang Feng
Redistribution prohibited without permission

The Score on the Plane: the Average Word, Two Poles, One Arrow

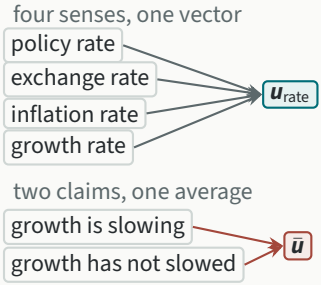


document	uncentred	centred
hike, tighten	$+0.94$	$+0.97$
cut, ease	$+0.54$	-0.96

- The same picture with numbers: nine hand-set points, μ their mean; the poles $\bar{u}_{\text{hawk}}$, $\bar{u}_{\text{dove}}$ are the seed lists' means; v joins them.
- *hike, tighten*: its arrow from μ runs nearly along v (dashed red, copied to μ), cosine $+0.97$; *cut, ease*: against it, -0.96.
- Uncentred, arrows start at the origin, where every word lies in v's quadrant (dashed grey): *cut, ease* scores $+0.54$, the wrong sign.
- **Centre first: a score is a direction relative to the average word.**

© Zhigang Feng
Redistribution prohibited without permission

Four Limits of One Vector per Word



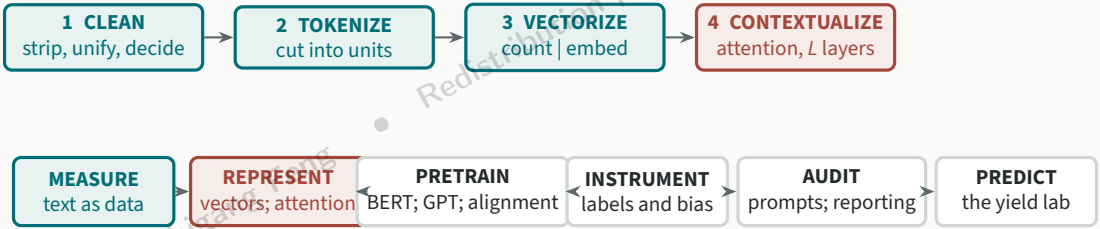
- **Polysemy.** *rate*: policy, exchange, inflation or growth; one vector averages the senses.
- **Negation.** "growth is slowing" and "growth has not slowed" average to one point.
- **Corpus dependence.** *Mask* meant face before 2020, mandate after: a language shift breaks a series.
- **Inherited bias.** man : programmer :: woman : homemaker (Bolukbasi et al. 2016; Caliskan et al. 2017).
- **All four come from one vector per word, whatever the sentence around it.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

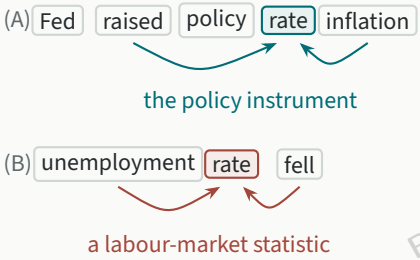
Step 4 · Contextualize

A weight computed from the sentence, instead of fixed before it is read



© Zhigang Feng
Redistribution prohibited without permission

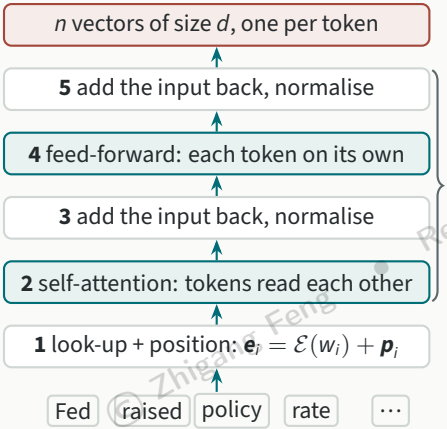
One Word, Two Sentences



- (A) The Fed raised its policy *rate* in response to persistent inflation. (B) The unemployment *rate* fell to its lowest level in decades.
- One token, two economic objects: an instrument the Fed sets, and a labour-market statistic.
- A static vector gives both the same $\boldsymbol{u}_{\text{rate}}$; a document vector with fixed ω cannot separate them.
- **The weights must be computed from the sentence itself.**

© Zhigang Feng
Redistribution prohibited without permission

The Transformer Layer at a Glance

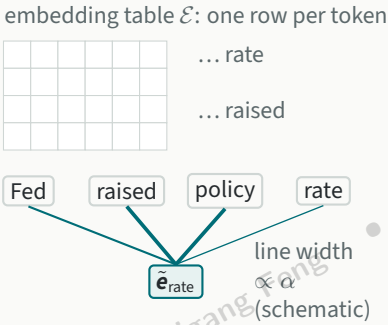


© Zhigang Feng
Redistribution prohibited without permission

- In: n tokens. Out: n vectors of size d, each now carrying its sentence.
- Only step 2 moves information between positions; the others work on one token's vector.
- Steps 3 and 5 keep each vector and rescale it; step 4 transforms it with one small shared network.
- **Steps 2–5 repeat L times (12 in multilingual BERT). The next frames open each box.**

The Idea: Re-Weight the Sentence for Each Word

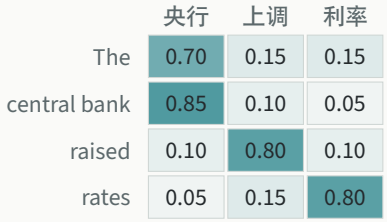
look-up + position → **self-attention** → add & normalise → feed-forward → add & normalise $\times L$



- The **embedding table** $\mathcal{E}$: one learned vector per token ID, plus a position vector: $\boldsymbol{e}_i = \mathcal{E}(w_i) + \boldsymbol{p}_i$.
- Rebuild token i: $\tilde{\boldsymbol{e}}_i = \sum_j \alpha_{ij}\, \mathcal{E}(w_j)$; w_j is token j, $\alpha_{ij} \geq 0$ its weight, $\sum_j \alpha_{ij} = 1$.
- The ω_j were fixed before reading; the α_{ij} come from *this* sentence: (A) *rate* leans on raised and policy, (B) on unemployment and fell.
- **The same token, different neighbours read, a different vector: self-attention.**

© Zhigang Feng
Redistribution prohibited without permission

Attention Began as Alignment in Translation

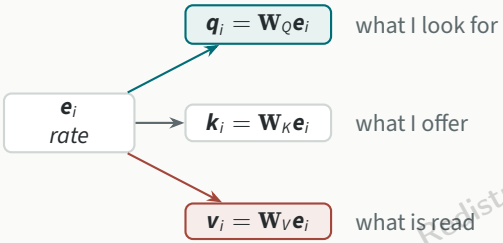


rows: the English word being written; columns: the Chinese source (central bank, raised, interest rate). Illustrative weights

- A translator who reads the whole Chinese sentence, closes the book and writes from memory: the recurrent network, one state holding everything.
- A translator who, for each English word, looks back at the source word that matters: attention (Bahdanau et al. 2015); the table is where the eye rests: **cross-attention**.
- The Transformer keeps the looking back and drops the memory state (Vaswani et al. 2017).
- **Self-attention is this looking back, turned on one sentence.**

© Zhigang Feng Redistribution prohibited without permission

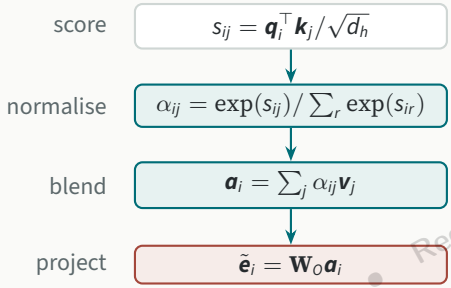
Query, Key, Value: Three Roles for One Token



- Three learned matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_h \times d}$ give each token three roles, request, index label, content: a soft database look-up.
- In (A) the query of *rate* meets the keys of Fed, raised, policy and inflation; in "the discount rates applied to pension funds", *rates* meets discount and pension.
- **Three separate matrices, so attention is asymmetric: *i* may read *j* heavily while *j* barely reads *i*.**

© Zhigang Feng
Redistribution prohibited without permission

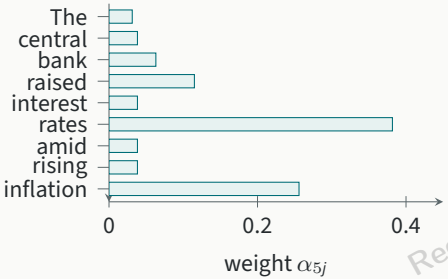
Score, Normalise, Blend, Project



- Sizes: $\boldsymbol{q}_i, \boldsymbol{k}_j, \boldsymbol{v}_j \in \mathbb{R}^{d_h}$; s_{ij} one number; $\mathbf{W}_O \in \mathbb{R}^{d \times d_h}$ returns $\tilde{\mathbf{e}}_i$ to size d.
- The softmax makes each row a distribution: position i spends one unit of attention on the value vectors.
- Why $\sqrt{d_h}$: the variance of $\boldsymbol{q}^\top \boldsymbol{k}$ grows with d_h; unscaled, the softmax piles onto one position and the gradients elsewhere vanish.
- **$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ and $\mathcal{E}$ are learned; α_{ij} is computed for every input: a share, never a coefficient.**

© Zhigang Feng
Redistribution not permitted without permission

Example 1: Scores to Weights for the Query *interest*



token j	s_{5j}	$e^{s_{5j}}$	α_{5j}
The	-0.2	0.82	0.03
central	0	1.00	0.04
bank	0.5	1.65	0.06
raised	1.1	3.00	**0.11**
interest	0	1.00	0.04
rates	2.3	9.97	**0.38**
amid	0	1.00	0.04
rising	0	1.00	0.04
inflation	1.9	6.69	**0.26**

Redistribution prohibited without permission
© Yingcai Feng

- s_{5j}: query token 5, *interest*, scored against token j, set by hand here ($\boldsymbol{q}_5^\top \boldsymbol{k}_j/\sqrt{d_h}$ once trained); $e^{s_{5j}}$ makes it positive; $\alpha_{5j} = e^{s_{5j}} / \sum_r e^{s_{5r}}$, its share of the sum, 26.13.
- **64% of the attention goes to *rates* and *inflation*: this *interest* is an interest rate, not a stake.**

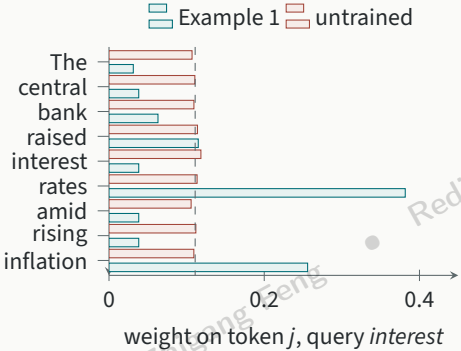
Example 2: Blend, Project, Add

		level	up	prices
rates	$0.38\times$	1	0.3	0
inflation	$0.26\times$	0	0.2	1
raised	$0.11\times$	0	1	0
rising	$0.04\times$	0	0.5	0.2
bank	$0.06\times$	0.5	0	0
$+\cdots=\boldsymbol{a}_5$		0.46	0.30	0.26
$\boldsymbol{e}_{\text{interest}}$		1	0	0
$\boldsymbol{e}+\tilde{\boldsymbol{e}}$		1.46	0.30	0.26

- Value vectors get three named coordinates: rate level, up, price pressure (illustrative; real ones are unnamed).
- Blend with Example 1's weights: $\boldsymbol{a}_5=\sum_j \alpha_{5j}\boldsymbol{v}_j=(0.46,\ 0.30,\ 0.26)$; with $\mathbf{W}_O=\mathbf{I}$ here, $\tilde{\boldsymbol{e}}_5=\boldsymbol{a}_5$.
- Add the token's own vector: $(1.0,\ 0.0,\ 0.0)+(0.46,\ 0.30,\ 0.26)=(1.46,\ 0.30,\ 0.26)$. Two *rate* tokens, identical before, now differ by what each read.
- **Same size, new content: *interest* now says "going up" and "price pressure".**

© Zhigang Feng
Redistribution prohibited without permission

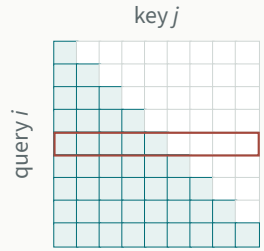
Example 3: The Same Arithmetic, Untrained



© Zhaoyang Cheng
Redistribution prohibited without permission

- The same tokens through an untrained head: random embeddings ($d = 16$) and random $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ ($d_h = 8$).
- Its row for *interest*: 0.106–0.118, against $1/9 = 0.111$. Nearly flat.
- Training (Deck 5.3) fits $\mathbf{W}_Q, \mathbf{W}_K$ so that the keys that matter score high. Lab C1 reproduces Example 1; C1b, this head.
- **The arithmetic is fixed; the pattern is learned.**

The Causal Mask: A Writer Must Not Read Ahead

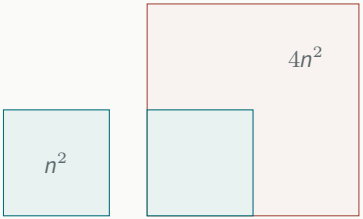


before	.03	.04	.06	.11	.04	.38	.04	.04	.26
masked	.11	.13	.22	.40	.13	0	0	0	0

- Goal: a generator writes token $i+1$ from tokens $1, \ldots, i$ (Deck 5.3). Trained unmasked, position i could read the answer and copy it.
- Set $s_{ij} = -\infty$ for $j > i$: $\alpha_{ij} = 0$, and each row renormalises over $j \leq i$: the **causal mask**.
- Example 1 masked: the sum falls to 7.47; *raised* takes 0.40; *rates* and *inflation* get 0. Lab C1b masks the untrained head.
- **An encoder reads a finished text: every token reads every other, the square. A decoder is writing: a token reads only what is on the page, the triangle.**

© Zhigang Feng · Redistribution prohibited without permission

What the Square Costs

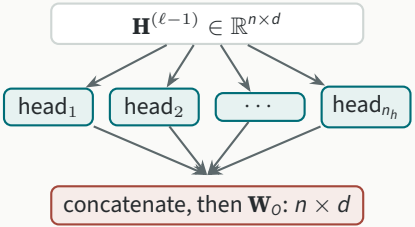


text	tokens n	scores n^2
the nine-token sentence	9	81
median FOMC statement	410	168,100
longest statement	1,053	1,108,809
100 statements	41,000	1,681,000,000

- Every query scores every key: n tokens give n^2 scores per head and layer, an $n \times n$ table (counts: the lab's 245 statements, GPT-4's tokenizer).
- Doubling n quadruples work and memory, so a long filing is cut into blocks that cannot see each other, severing Item 7 from Item 1A: Deck 5.3's window limit.
- **The square is the price of letting every token read every other.**

© Zhigang Feng Redistribution prohibited without permission

Many Heads, and the Whole Sentence at Once



$4n_h d_h d = 4d^2$ parameters, whatever n_h is

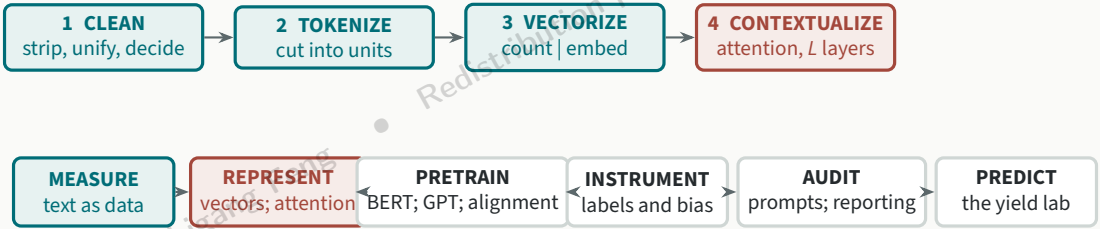
- Rows $\mathbf{H} \in \mathbb{R}^{n\times d}$. Head h has its own $\mathbf{W}_Q^h, \mathbf{W}_K^h, \mathbf{W}_V^h$, so $\mathbf{Q}_h = \mathbf{H}\mathbf{W}_Q^h$ etc., and computes $\mathrm{softmax}(\mathbf{Q}_h\mathbf{K}_h^\top/\sqrt{d_h})\,\mathbf{V}_h$, $d_h = d/n_h$.
- Why they differ: different random starts, then training moves each to the pattern that lowers the loss the others miss. Nothing assigns the jobs.
- Readers of one set of minutes: inflation, the labour market, references back.
- **A head weights positions; it does not split a document into topics.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.2

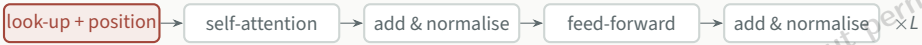
One Transformer Layer

Position, residual connections, normalisation, and the feed-forward block



© Zhigang Feng
Redistribution prohibited without permission

Attention Cannot See Word Order



the same tokens, opposite claims

growth exceeded inflation

inflation exceeded growth

queries and keys see tokens, not places

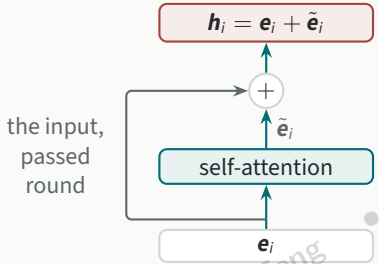
$$\boldsymbol{e}_i = \mathcal{E}(w_i) + \boldsymbol{p}_i$$

- Without position vectors attention is permutation-invariant: shuffle the tokens and no score changes (a causal mask leaks some order; Haviv et al. 2022).
- **Sinusoidal**: $p_{i,2l} = \sin(i/10000^{2l/d})$, $p_{i,2l+1} = \cos(i/10000^{2l/d})$.
- Learned positions stop at the training length; RoPE (Su et al. 2024) with scaling and ALiBi (Press et al. 2022) read past it.
- **Order is not in the mechanism; it is added to the input.**

© Zhigang Feng
Redistribution prohibited without permission

Residual Connection: Keep the Token, Add What It Read

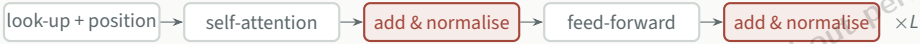




- Output = input + update: the sum keeps the word as well as what it read. “Residual” names the update, not a regression residual or Session 2’s Euler residual.
- $\partial \boldsymbol{h}_i / \partial \boldsymbol{e}_i = \mathbf{I} + \partial \tilde{\boldsymbol{e}}_i / \partial \boldsymbol{e}_i$: the $\mathbf{I}$ carries the gradient through all L layers; without it one Jacobian per layer shrinks it, the **vanishing gradient**.
- **A layer with nothing to add learns $\tilde{\boldsymbol{e}}_i \approx 0$ and passes its input on.**

© Zhigang Feng
Redistribution prohibited without permission

Layer Normalisation: Standardise Each Token's Vector



one token's vector, $d = 4$

2	4	6	8

$(\boldsymbol{x} - \mu)/\sigma$

$\mu = 5, \sigma = 2.24$

-1.34	-0.45	0.45	1.34

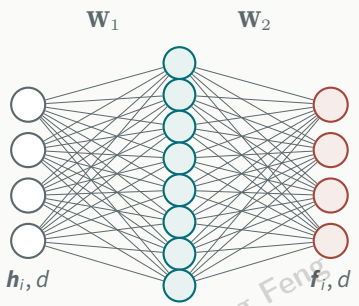
then $\boldsymbol{\gamma} \odot (\cdot) + \boldsymbol{\beta}$, learned per coordinate

- $\mathrm{LN}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot (\boldsymbol{x} - \mu)/\sqrt{\sigma^2 + \epsilon} + \boldsymbol{\beta}$: μ, σ over the d entries of *one* token's vector; $\boldsymbol{\gamma}, \boldsymbol{\beta}$ learned.
- Why: each residual adds to the vector, so its scale drifts layer by layer; LN puts every layer's input on one scale, like standardised regressors.
- The strip and the layer picture show the original order, add then normalise; Machine B and most current models normalise before each block.
- **Neither the residual nor LN moves information between tokens. Only attention does.**

© Zhigang Feng
Redistribution prohibited without permission

The Feed-Forward Block: Transform Each Token on Its Own

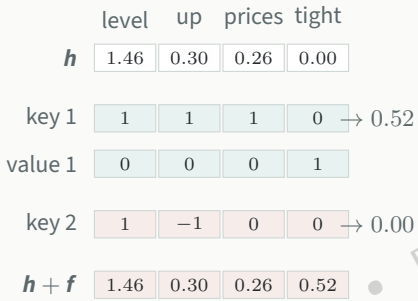
look-up + position → self-attention → add & normalise → **feed-forward** → add & normalise $\times L$



- Attention only averages values, a linear mix; combining *rate*, *raised*, *inflation* into "tightening" needs a nonlinear step.
- $\boldsymbol{f}_i = \mathbf{W}_2\,\mathrm{ReLU}(\mathbf{W}_1\boldsymbol{h}_i + \boldsymbol{b}_1) + \boldsymbol{b}_2$, $\mathbf{W}_1 \in \mathbb{R}^{4d\times d}$, $\mathbf{W}_2 \in \mathbb{R}^{d\times 4d}$: Session 2's one-hidden-layer network.
- Same weights at every position; output size d, added back. Machine B uses GELU, a smooth ReLU.
- **Attention moves information between tokens; this block transforms each one.**

© Zhigang Feng Redistribution prohibited without permission

Example: A Hidden Unit as a Key–Value Memory



© Zhigang Feng

Redistribution prohibited without permission

- Illustrative coordinates: rate level, up, price pressure, tightening; ***h*** from Example 2; bias -1.5.
- Unit 1's key (a row of $\mathbf{W}_1$): $2.02 + (-1.5) = 0.52$. It fires on "a rate going up with price pressure" and adds $0.52\times$ its value (a column of $\mathbf{W}_2$).
- Unit 2 looks for "a rate going down": -0.34, ReLU 0, silent. The vector now says "tightening", which no token said.
- **Rows of $\mathbf{W}_1$ detect patterns; columns of $\mathbf{W}_2$ write back what goes with them (Geva et al. 2021).**

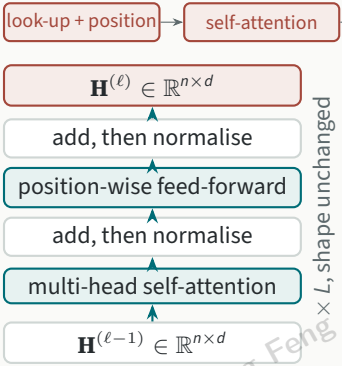
Where the Parameters Are: Machine B, Part by Part

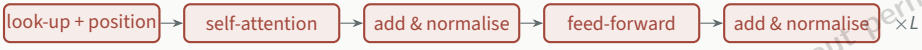
part of Machine B	count	share
feed-forward $\mathbf{W}_1$, 3 layers	198,144	31.4%
feed-forward $\mathbf{W}_2$, 3 layers	196,992	31.2%
attention $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$, 3 layers	148,608	23.5%
attention $\mathbf{W}_O$, 3 layers	49,536	7.8%
position table, 128 positions	16,384	2.6%
character table, 78 characters	9,984	1.6%
output layer, 78 characters	9,984	1.6%
layer norms before attention	768	0.1%
layer norms before feed-forward	768	0.1%
final layer norm	256	0.0%
whole model	631,424	100%

- **Machine B**: Session 1's character-level text generator, trained on the 245 FOMC statements less every 20th; 3 layers, 4 heads, $d = 128$.
- Per layer, feed-forward $8d^2$ against attention's $4d^2$: 62.6% of the model. Lab C2 prints this table.
- **Most parameters sit in the feed-forward blocks, where associations learned from text are kept.**

© Zhigang Feng
Reproduction prohibited without permission

One Layer, Stacked L Times

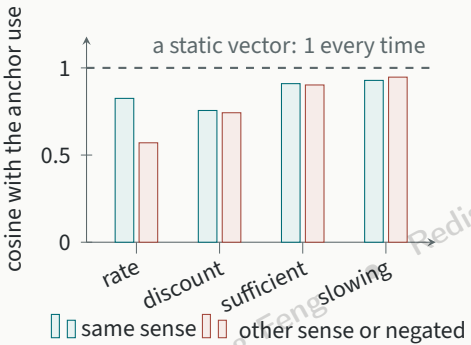




- The layer's output has the input's shape, $n \times d$, so layers stack with no change of interface.
- Shallow layers lean to local syntax (*quantitative easing*); deep ones link paragraphs, a "risk factor" early in an annual report with a "regulatory change" later.
- Machine B stacks 3 layers, $d = 128$; multilingual BERT 12, $d = 768$.
- **$\mathbf{H}^{(L)}$ is where Deck 5.3 starts.**

© Zhigang Feng · Redistribution prohibited without permission

What Context Buys, Measured



© Zhigang Feng — Redistribution prohibited without permission

- Multilingual BERT (12 layers, $d = 768$), last layer: one word in three sentences, cosine with its anchor use.
- *rate* separates its senses, 0.825 vs 0.570 (unemployment rate); *discount* barely, 0.756 vs 0.743 (a price discount).
- Negation barely moves it: *sufficient* 0.910 vs 0.902; for *slowing* the negated use is the closer, 0.947 vs 0.928.
- **Context can fix a sense; whether it does, and whether it reads "not", must be measured.**

Reading 10-Ks by Paragraph Attention: Design and Findings

one 10-K filing, split into paragraphs

↓

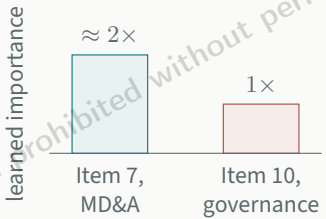
embed each: 3,072 numbers; keep 64

↓

self-attention over paragraphs, then pool

↓

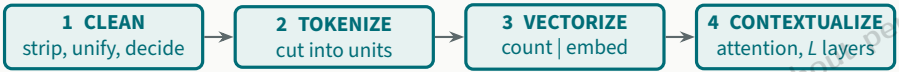
sign of the $[-1, +30]$ return



© Zhigang Feng. No distribution or reproduction prohibited without permission

- Kim, Muhn, Nikolaev & Zhang (2024) make the paragraph the "token", one vector each: 76,929 filings, 2004–2024, 20,712,462 paragraphs.
- Target: the sign of the return from a day before the filing to 30 days after. Attention beats both attention-free baselines; MD&A gets almost twice governance's weight.
- Learned to predict returns, so "important" means useful, not causal; template text repeated in every filing may spread α_{ij} thin.
- **Attention routes information inside the text it is given. It does not retrieve truth.**

The Whole Pipeline



rung	"rates" becomes	size	similar words	senses	you can read
counts, TF-IDF	one column of the matrix	$V \sim 10^4$–10^5, mostly zeros	orthogonal	one per word	each axis is a word
word vector	one point $\boldsymbol{u} \in \mathbb{R}^d$	$d = 100$–300, dense	close	one per word	no axis is a word
contextual vector	a point per sentence	$d = 768$ in multilingual BERT	close	one per use	the weights, not the meaning

© Zhitang Feng
Redistribution prohibited without permission

- Report every step: the cleaning script; the tokenizer and its version; the vector method, corpus and period; the model and its version.
- **Each rung buys meaning and sells the audit chain: pick the rung the design needs.**

Lab, Parts B and C: Vectors and Attention on the FOMC Statements

cell	what you run	the question you answer
B1	word vectors by PPMI and SVD	Which neighbours are phrases? Which survive another window?
B2	the hawk–dove direction	Which seeds survive, and what does the score bundle in?
B3	validation on the policy path	Where does the embedding score fail?
C1, C1b	Example 1; the untrained head, masked	What does the mask set to 0? How flat is the row?
C2–C3	Machine B's parameters and maps	Where are the parameters? Is the upper triangle 0?



- **Each cell reproduces the slide numbers it names; C4–C5 belong to Deck 5.3.**

© Zhaoning Feng · Redistribution prohibited without permission

Session 5.3

Pretrained Language Models: Encoders That Read, Decoders That Write

Two routes from the Transformer to a measurement: what each was built for, how it works, how its weights are obtained, and what economists measure with it

Based on Chapter 13 of Miao & Feng, *AI for Economists*

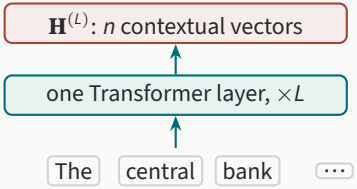
Redistribution prohibited without permission



© Zhigang Feng

Zhigang Feng

Where the Pipeline Stands

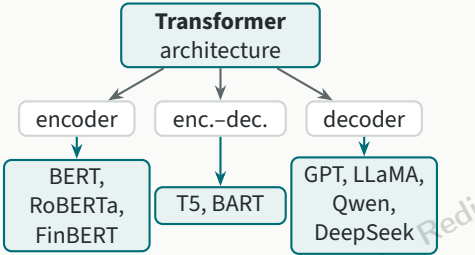


untrained, the attention row for *interest* runs 0.106–0.118: flat

- Deck 5.2 stacked one Transformer layer L times: n tokens in, n contextual vectors $\mathbf{H}^{(L)}$ out. With random weights it reads nothing.
- This deck: two model families, each in four steps: what it was built for, how it reads or writes, how its weights are obtained, what economists measure with it.
- **Pretraining puts language into the weights; how you use them decides the measure.**

© Zhigang Feng
Redistribution prohibited without permission

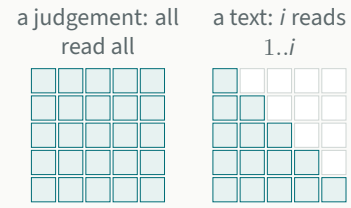
The Family Tree: Architecture, Pretrained Model, Product



© Zhigang Feng · Redistribution prohibited without permission

- **Architecture**: Deck 5.2's wiring with random weights. **Pretrained model**: the same wiring, weights fit on a corpus; encoders read, decoders write.
- **Product**: a pretrained decoder taught to follow instructions (**alignment**) and served with default settings.
- **Name the layer you mean: the product is not the architecture.**

Three Jobs, Three Wirings



- The question: what should come back from the model? A judgement about the text, a text, or another sequence.
- A judgement (a label, a score, a vector) → **encoder**: every token reads every other, the square; BERT. A text → **decoder**: a token reads only what is already written, the triangle; GPT.
- Another sequence (a translation; {action: raise, reason: inflation}) → **encoder–decoder**: T5, BART. Measurement tasks fall on the first two.
- **The job fixes the wiring, and the wiring fixes what the model can be asked for.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.3

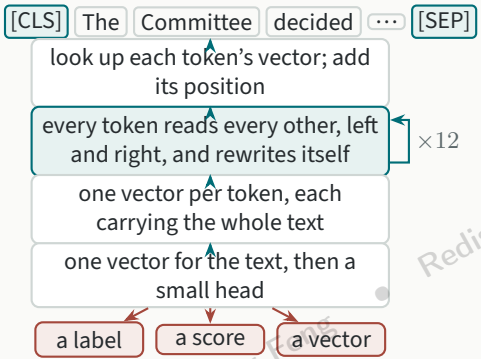
The Encoder Route

Built to read: what BERT is for, how it reads, how its weights are obtained, what it measures



© Zhigang Feng Redistribution prohibited without permission

What an Encoder Is Built For: BERT in Plain Words

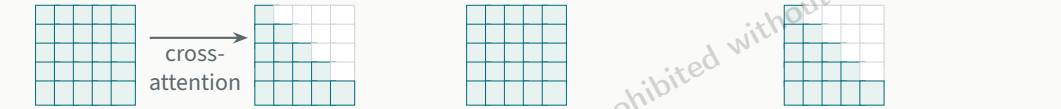


© Zhigang Feng

Redistribution prohibited without permission

- BERT reads: text in, numbers out, never more text. A token starts as its word vector plus its position.
- Twelve rounds, no mask: every token reads every other, left and right, and rewrites its vector; at the end each carries the whole text.
- One vector for the text, then a small head: a label, a score, or the vector itself. Weights from guessing hidden words (next frames); the head from a few hundred labels.
- **Text in, numbers out, for a decision made elsewhere: the measurement workhorse.**

How BERT Differs From the Transformer: the Reading Half, Kept Alone

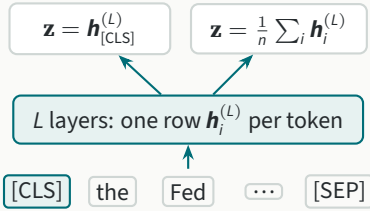


	Transformer, 2017	BERT	GPT, Machine B
reads	square, then triangle	the square: left and right	the triangle: left only
trained to	translate	fill in hidden words	predict the next token
returns	a translation	a judgement	a next-token distribution

- The 2017 Transformer translated: an encoder read the source, a decoder wrote the target, looking back at it (Vaswani et al. 2017).
- BERT keeps the encoder alone, twelve of the same layers, no mask: it reads, never writes (Devlin et al. 2019). GPT keeps the decoder alone: it writes (Radford et al. 2018).
- **Same layer inside; who reads whom, the training task and the output differ.**

© Zhiwei Feng Redistribution prohibited without permission

How an Encoder Reads: Both Sides, Then One Vector



cosine of passage vectors	mean	[CLS]
A and B (both on rising rates)	0.72	0.91
A and an unrelated sentence	0.46	0.83
B and an unrelated sentence	0.46	0.85

- Every token reads every other, left and right: the square. A blank needs both sides: *raised*? *cut*?
- Input [CLS] $w_1 \ldots w_n$ [SEP]: [CLS] is a slot with no word whose last row stands for the text; [SEP] marks the end.
- **Pooling**, two ways to one vector: the [CLS] row, or the mean of all rows. Table: a policy statement (A) and an annual-report passage (B), recorded.
- **Untuned, pool by the mean (0.72 related, 0.46 not); raw [CLS] rows are all alike until a task trains them.**

© Zhigang Feng Redistribution prohibited without permission

How the Weights Are Obtained, Stage 1: Fill in the Blanks



both sides needed: *raised*? *cut*?

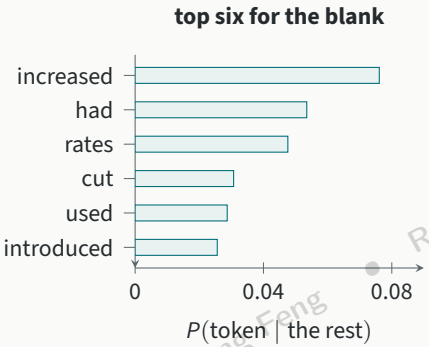
choose 15% of positions; of those 80% → [MASK], 10% → a random token, 10% unchanged

BERT also predicted whether sentence B follows A; RoBERTa found that dropping it "matches or slightly improves" results (Liu et al. 2019).

- Goal: fit the weights on raw text, no labels: hide words and let the text supply the answers (Devlin et al. 2019).
- $\mathcal{M}$: the chosen 15% of positions; $\tilde{w}_{1:n}$: the corrupted input. Maximise $\sum_{i \in \mathcal{M}} \log P_\theta(w_i \mid \tilde{w}_{1:n})$, $P_\theta = \mathrm{softmax}(\mathbf{W}_{\text{out}} \boldsymbol{h}_i^{(L)})$ over the V tokens.
- The token [MASK] never appears in use, hence the 10% kept and the 10% swapped.
- **To fill a blank the model must read both sides, so $\mathbf{H}^{(L)}$ encodes context.**

© Zhigang Feng · Redistribution prohibited without permission

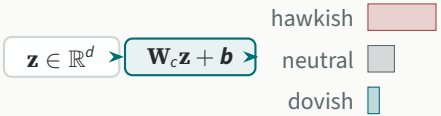
Example: A Pretrained Encoder Fills the Blank



- Multilingual BERT's masked-word head, recorded offline, on "The central bank [MASK] interest rates amid rising inflation.": *increased* 0.076, *cut* 0.031, the true *raised* 0.019; this blank's loss is $-\log 0.019 = 3.97$.
- "Inflation (unemployment) is rising, so the Committee decided to [MASK] the target range": $P(\textit{raise})$ 0.0014 vs 0.0029, the wrong way round.
- **Pretraining delivers the kind of word, not the economics: hence domain text and labels.**

© Zhigang Feng
Redistribution prohibited without permission

Stage 2: A Head on Labels

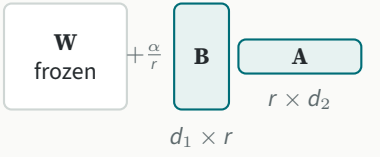


softmax over $C = 3$ classes (bars schematic)

- Goal: a stance for each sentence, learned from N hand-labelled ones.
- Head: $\boldsymbol{p} = \mathrm{softmax}(\mathbf{W}_c\mathbf{z} + \boldsymbol{b})$, $\mathbf{W}_c \in \mathbb{R}^{C \times d}$; loss $-\frac{1}{N}\sum_i \log p_{i,y_i}$: a multinomial logit on the encoder's features.
- Counts: the head has $C(d+1) = 2,307$ numbers ($d = 768$); the encoder 177,853,440. Full fine-tuning moves the encoder too.
- **A few hundred labels can pin down the head, not the encoder.**

© Zhigang Feng
Redistribution prohibited without permission

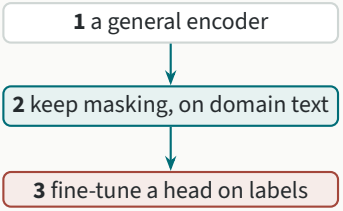
Three Ways to Fine-Tune



- **Full**: update all 177,853,440 weights. Closest fit; with few labels it overfits and loses the general representation.
- **Frozen encoder**: train the head alone. Cheap; the representation cannot adapt.
- **Low-rank adaptation** (LoRA; Hu et al. 2022): $\mathbf{W}' = \mathbf{W} + \frac{\alpha}{r}\mathbf{BA}$, $\mathbf{A}$ random, $\mathbf{B} = \mathbf{0}$ at the start, so training begins at the pretrained model. At $d_1 = d_2 = 768$, $r = 8$: 12,288 numbers against 589,824, 2.1%.
- **The change a task needs has low rank: one base model, one small versioned adapter per task.**

© Zhigang Feng
Redistribution prohibited without permission

Domain-Adaptive Pretraining: Same Objective, Different Corpus



- Continue the masking objective on domain text before adding a head (Gururangan et al. 2020): *hawkish* is a stance on war in general text, on rates in central-bank text.
- FinBERT (Araci 2019): BERT, then masking on TRC2-financial (46,143 Reuters articles, over 29M words), then a three-way head on the Financial PhraseBank (4,845 sentences).
- **Which corpus the model read is part of what the measure means.**

© Zhigang Feng
Redistribution prohibited without permission

Encoders in Economic Research

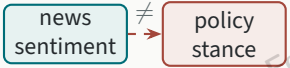
read text, return …	model	what the number means
news sentiment, per sentence	FinBERT (Araci 2019): a three-way head on the Financial PhraseBank	positive, negative or neutral financial news
agent and sign, per central-bank sentence	CentralBankRoBERTa (Pfeifer & Marohl 2023): two classifiers	which of five agents a sentence concerns, and whether it is good for them
hawkish–dovish stance	a head fine-tuned on hand-coded sentences (Deck 5.4)	the coders' definition of stance, carried to every sentence
the class of a 10-K risk paragraph	a head with classes such as regulatory, market, operational	the class list the researcher wrote
a vector, for similarity	PaECTER, an encoder trained on patents (Ghosh et al. 2024); Session 6's index	the cosine of pooled vectors: nearness, not sameness

© Zhigang Feng. Redistribution prohibited without permission

- **Every row is one head or one cosine on $\mathbf{H}^{(L)}$, the same number at every run; what it means is the training labels and the corpus.**

Example: A Domain Model's Labels Are Its Definition

sentence, written for this slide	positive	negative	neutral
The Committee decided to raise the target range for the federal funds rate.	0.12	0.03	**0.86**
The Committee decided to lower the target range for the federal funds rate.	0.05	**0.50**	0.45
The Committee decided to maintain the target range for the federal funds rate.	0.06	0.03	**0.91**
The labor market has remained strong, and job gains have been solid.	**0.95**	0.03	0.02
Economic growth has slowed, and business investment has weakened.	0.01	**0.97**	0.02



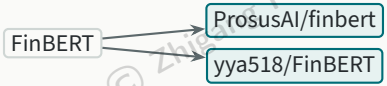
- `ProsusAI/finbert`, recorded offline. A hike reads neutral (0.86), a cut negative (0.50): news for markets, not policy stance.
- **A classifier measures what its training labels meant: read the label set first.**

© Zhigang Peng
Redistribution not allowed without permission

Two FinBERTs, and a Model With Two Classifiers

identifier	its paper	what it is
`ProsusAI/finbert`	Araci (2019)	financial news, then a three-way head on the Financial PhraseBank
`yya518/FinBERT`	Yang, UY & Huang (2020)	a different model of the same name, on financial communications
CentralBankRoBERTa	Pfeifer & Marohl (2023)	*two* classifiers: five macroeconomic agents, and a sign for each

© Zhaoyi Feng
Redistribution prohibited without permission



one name, two models

- **Load a model by its identifier, cite the paper with it, and read its label set.**

Session 5.3

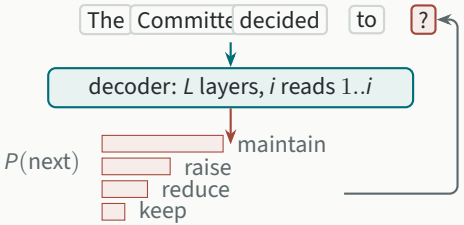
The Decoder Route

Built to write: what GPT is for, how it writes, how its weights are obtained, what it measures



© Zhigang Feng Redistribution prohibited without permission

What a Decoder Is Built For

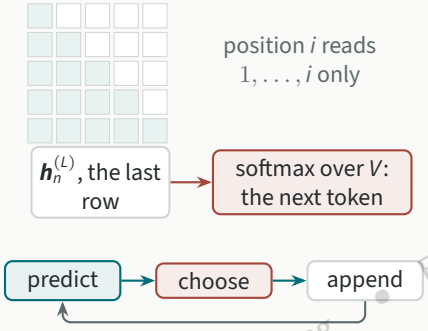


schematic. Choose one, append it, ask again

- Built for writing: given a prefix, a probability for every vocabulary token as the next one (GPT, Radford et al. 2018).
- With scale the purpose grew (Radford et al. 2019; Brown et al. 2020): write a task as text to be continued, “Stance:”, and one model serves many tasks with no head to fit.
- In: a prefix. Out: one distribution. Choose a token, append it, repeat: a text.
- **A decoder turns a prompt into text; measuring with it means asking in the right form.**

© Zhigang Feng
Redistribution prohibited without permission

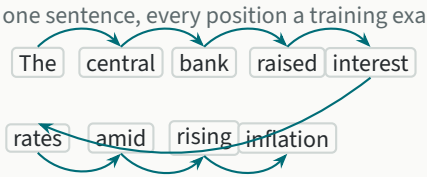
How a Decoder Writes: One Row, One Distribution, One Loop



- The encoder's blocks with the causal mask: each row sees only its past. The last row, through the output layer: $P_\theta(w_{n+1} \mid w_{1:n}) = \mathrm{softmax}(\mathbf{W}_{\text{out}} \boldsymbol{h}_n^{(L)})$.
- Generation loops: predict, choose one token, append it, predict again. **Greedy**: the top token.
- **Temperature** τ divides the scores s_w before the softmax: $\tau \to 0$ is greedy, $\tau > 1$ flattens. Top-k, top-p: draw from the best k, or the smallest set with mass p.
- **The model returns a distribution; a decoding rule turns it into text.**

© Zhigang Feng · Redistribution prohibited without permission

How the Weights Are Obtained: Predict the Next Token



8 predictions from nine tokens:
The→central, …, rising→inflation.
Masking 15% of the same sentence gives about 1.35.

- Objective: maximise $\sum_i \log P_\theta(w_i \mid w_{<i})$ over the corpus: at every position, the probability of the token that actually came next (**causal language modelling**).
- No labels: the text supplies its own answers, as the blanks did for the encoder; here every position is an example, not 15% of them.
- The price: only the left context, so position i never reads what follows it.
- **A denser training signal and one-sided reading: that trade separates GPT from BERT.**

© Zhigang Feng
Redistribution prohibited without permission

Example: A Next-Token Distribution, Recorded

next character	P	score
`m`	0.532	6.34
`r`	0.084	4.49
`k`	0.072	4.35
`t`	0.061	4.18
`i`	0.049	3.95
`a`	0.038	3.69

The Committee decided to␣ m ?

- **Machine B**: Session 1's text generator, trained on the 245 FOMC statements; 3 layers, $d = 128$, 631,424 parameters, 78 characters as tokens.
- After "The Committee decided to" it puts 0.53 on `m` (maintain), then `r` (raise, reduce) and `k` (keep). The top character each time: "… to maintain the target range for the federal funds rate at 0 to".
- **A fitted decoder is this table at every step; the score column is what temperature rescales. Lab C4 runs it.**

© Zhigang Feng
Redistribution prohibited without permission

Example: Decoding Rules on Machine B's Scores

next	$\tau = 0.5$	$\tau = 1$	$\tau = 2$
`m`	0.93	0.53	0.19
`r`	0.02	0.08	0.07
`k`	0.02	0.07	0.07
`t`	0.01	0.06	0.06

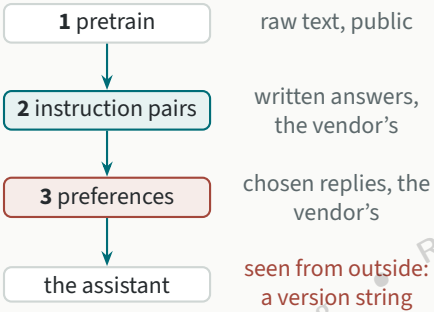
$p_\tau(w) \propto \exp(s_w/\tau)$, the last frame's

$\tau \to 0$ greedy | $\tau = 1$ the model | $\tau > 1$ flatter

- Columns: the same scores divided by τ before the softmax. At $\tau = 0.5$ the top character takes almost everything; at $\tau = 2$ the mass spreads.
- Sampled at $\tau = 2$, Machine B writes "neiqvelopments, both polifle advi2$105. Biral 2 1.S5 fillp4": the spelling of the corpus, the sense gone.
- **For measurement set $\tau = 0$, and report τ, the seed, the version and an input hash.**

© Zhiqiang Feng Redistribution prohibited without permission

Stage 2 in One Frame: From Continuation to Answers



- A pretrained decoder continues text: asked a question, it may go on with another question (illustrative). Fluent, not an answer.
- Stage 2: the next-token objective again, on instruction–answer pairs people wrote. It fixes form: format, and when to decline.
- Stage 3: people see two replies and pick one; the model moves toward the preferred kind, kept close to stage 2 (Ouyang et al. 2022).
- **Both data sets are private and rolling: a chat product is a moving instrument, and the version string is part of the measure.**

© Zhigang Feng · Redistribution prohibited without permission

Decoders in Economic Research

read text, write …	example	what you must report
a label, zero- or few-shot	Hansen & Kazinnik (2023): GPT-4 labels sentences of the 2010–2020 FOMC statements hawkish, neutral or dovish; matched human coders better than a fine-tuned encoder; moved with prompt wording and version	the prompt, the version, agreement with a hand-coded sample (Deck 5.4)
fields, from raw text	dates, parties, amounts from prospectuses, court rulings or job postings, into a panel	the schema, and a checked subsample
a score for a construct	policy uncertainty or financing pressure, one number per document from a rubric	the rubric, $\tau = 0$, several raters (Decks 5.4–5.5)
answers in a persona	a model filling in a survey as a household or a firm	Deck 5.5

© Zhigang Feng ■ Redistribution prohibited without permission

- **Every row is a continuation of a prompt, no parameter changed; so the measure is the prompt, the decoding rule and the version.**

In-Context Learning: The Task Goes in the Prompt

Label the monetary-policy stance of the Federal Open Market Committee statement below. Answer with exactly one word: HAWKISH if it leans toward tighter policy, DOVISH if it leans toward easier policy, NEUTRAL if it leans neither way. Output the single word and nothing else.

STATEMENT: [the statement of 1995-02-01: "The Federal Reserve Board today approved an increase in the discount rate from 4 3/4 percent to 5 1/4 percent, effective immediately." …]

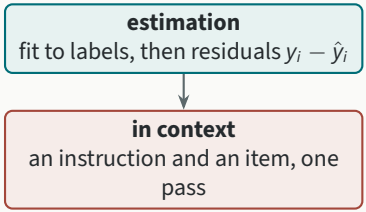
continuation → HAWKISH

recorded: claude-sonnet-5, 2.1.278 (Claude Code), 2026-09-20

- **Zero-shot**: an instruction and the item, as here. **Few-shot**: add worked examples first, e.g. "… decided to raise the target range. HAWKISH" (written for this slide).
- No parameter changes: the label is the model's continuation of the prompt.
- **The prompt is the estimator's specification: freeze it and publish it.**

© Zhuang Feng — redistribution prohibited without permission

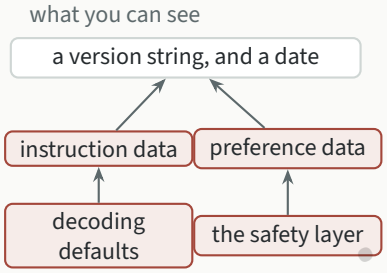
In-Context Labels Carry No Internal Error Measure



- Estimation fits parameters to labelled data and leaves regression residuals $y_i - \hat{y}_i$ to inspect.
- In-context labelling fits nothing: the model returns HAWKISH and no quantity says how likely it is to be wrong.
- Machine B's fluent statements had the same property: no target to subtract them from.
- **So the error is measured outside, on a hand-coded sample. Deck 5.4 builds it.**

© Zhigang Feng
Redistribution prohibited without permission

The Version Is the Instrument



- Instruction and preference data are the vendor's, private and rolling; from outside you see a version string, `gpt-4-0613` or `gpt-4o-2024-08-06`.
- Two studies that both "used GPT-4" may have used different instruments, and neither can tell. Deck 5.5 scores one frozen rubric with several raters.
- **An aligned model used for scoring is a time-varying instrument: the version is part of the measure's definition.**

© Zhigang Feng
Redistribution prohibited without permission

Session 5.3

Measuring With Either Family

The comparison, the limits that stay, and the lab



© Zhigang Feng
Redistribution prohibited without permission

Two Paragraphs, Two Families

A, a policy statement: To counter rising inflation pressure, the central bank decided to raise the policy rate by 25 basis points. The Committee judges that maintaining a restrictive stance is appropriate until inflation returns to its target.

B, an annual report: During the reporting period, the company's financing costs increased as market interest rates rose. Management expects tighter credit conditions to put some pressure on next year's capital expenditure.

© Anirang Zeng · Redistribution prohibited without permission

- **BERT**: pool each into $\mathbf{z}_A, \mathbf{z}_B$: cosine 0.72, 0.46 against unrelated text.
- **GPT** (illustrative): "… decided to" continues *raise*; asked A's stance, "hawkish"; asked B's risk type, {risk type: rising financing costs}.
- **BERT returns the same number every time; GPT moves with the prompt, decoding and version.**

Five Limits, Five Architectural Roots

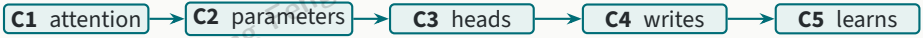
limit	root in the architecture	what it does to a design
the cutoff	knowledge sits in the weights, fixed on a date	put the document in the prompt
hallucination	the objective rewards form, not truth	check every citation and number
the window	attention costs $O(n^2)$ (Deck 5.2)	blocks sever cross-references
lost in the middle	attention favours the ends	position is part of the error
prompt sensitivity	the prompt is the prior	freeze it, and vary the order

© Zhirang Feng · Redistribution prohibited without permission

- None is a defect awaiting a patch; each follows from how the machine is built (the middle: Liu et al. 2024; the order: Brucks & Toubia 2025).
- **Deck 5.4 turns them into a protocol; Session 6 repairs four by retrieval.**

Lab, Parts C4–C5: Machine B Writes, Then Learns From Nothing

cell	what you run	the question you answer
C4	Machine B writes 200 characters after “The Committee decided”, at three temperatures	Which dial moves the house phrasing, and which the spelling? What did the objective never ask for?
C5	a fresh model of the same architecture, sixty training steps on the same corpus, held out by whole statement	Where does the loss start ($\ln 78$ nats per character), and what has it learned after sixty steps?



- **C4 reproduces the next-character and temperature tables; C5 starts where every model starts, at the uniform distribution.**

© Zhigang Feng
Redistribution prohibited without permission